# Proyecto Final




# Celia Morán Gámez

1. Introducción ..... 5
Objetivos principales ..... 5
2. Requisitos ..... 6
Requisitos específicos ..... 6
Requisitos del sistema ..... 6
Requisitos previos ..... 6
Herramientas necesarias ..... 7
Descargar el código fuente ..... 7
3. Análisis ..... 8
Casos de uso ..... 8
Diagrama de clases ..... 10
Diagramas de secuencia ..... 11
Diagrama de secuencia Iniciar sesión ..... 11
Diagrama de secuencia Mostrar anuncio específico ..... 12
Diagrama de secuencia Mostrar publicaciones ..... 13
Diagrama de secuencia Publicar anuncio ..... 14
Diagrama de secuencia Dar favorito ..... 15
Diagrama de secuencia Registro ..... 16
4. Diseño ..... 17
Capa de presentación ..... 18
Capa Lógica ..... 24
Capa de datos ..... 29
Tabla Anuncio ..... 29
Tabla usuarios ..... 30
Tabla anuncios_favoritos_usuarios ..... 31
Tabla anuncio_fotos ..... 31
5. Ejemplo de implementación ..... 32
Parte pública ..... 32

Home Page ..... 32
Anuncios Page ..... 35
Anuncio Page ..... 37
Inicio sesión Page ..... 39
Registro Page ..... 40
Parte privada ..... 41
Mi perfil Page ..... 41
Mis Publicaciones Page ..... 44
Publicar anuncio Page ..... 45
Not Found Page ..... 49
6. Mejoras ..... 50
Búsqueda por mapa ..... 50
Simular hipoteca ..... 50
Noticias ..... 50
Optimizar el front-end ..... 51
7. Recursos ..... 52
Figma ..... 52
Postman ..... 54
MySQL Workbench ..... 55
Spring Tool Suite 4 ..... 55
Visual Studio Code ..... 56
GitHub ..... 56
Angular Material ..... 56

# 1. Introducción

## Objetivos principales

El principal objetivo de este proyecto será el desarrollo e implementación de un portal web de gestión de anuncios inmobiliarios. Las funciones serán las siguientes:

Principales funciones:

- Iniciar sesión del usuario existente.
- Registro de un nuevo usuario.
- Todos los usuarios (se hayan o no registrado) podrán acceder al portal de visualización de todos los anuncios. Ahí podrán ver todos los anuncios publicados y disponibles actualmente.
- Filtrar búsqueda según el tipo de inmueble.
- Compartir una publicación.

Funciones para usuarios registrados únicamente:

Estas serán funciones que solamente se podrán realizar si el usuario ha iniciado sesión previamente:

- Guardar como favorita una publicación.
- Visualizar mis publicaciones guardadas en favoritos.
- Publicar un nuevo anuncio.
- Despublicar un anuncio.
- Visualizar mis propios anuncios publicados y los despublicados.

# 2. Requisitos

## Requisitos específicos

A continuación, se describen los requisitos específicos del proyecto, que se dividen en funcionalidades de la parte pública y la parte privada. La parte pública está disponible para todos los usuarios, sin importar si han iniciado sesión o no, mientras que la parte privada requiere autenticación previa para acceder a funciones avanzadas y personalizadas.

Parte pública:

- Iniciar sesión del usuario existente.
- Registro de un nuevo usuario.
- Todos los usuarios (se hayan o no registrado) podrán acceder al portal de visualización de todos los anuncios. Ahí podrán ver todos los anuncios publicados y disponibles actualmente.
- Filtrar búsqueda según el tipo de inmueble.
- Compartir una publicación.

Parte privada:

Estas serán funciones que solamente se podrán realizar si el usuario ha iniciado sesión previamente:

- Guardar como favorita una publicación.
- Visualizar mis publicaciones guardadas en favoritos.
- Publicar un nuevo anuncio.
- Despublicar un anuncio.
- Visualizar mis propios anuncios publicados y los despublicados.

## Requisitos del sistema

Este proyecto no cuenta con un dominio público ni está alojado en un servidor, por lo que para ejecutarlo es necesario descargar el código fuente y configurar el entorno de desarrollo. A continuación, se detallan los pasos y requisitos para acceder al proyecto.

### Requisitos previos

- Tener un ordenador con acceso a internet
- Sistema operativo compatible (Windows, macOS o Linux)

## Herramientas necesarias

- Spring Tool Suite 4 (STS 4): IDE basado en Eclipse para desarrollar el backend con Spring Boot.
- Visual Studio Code: Editor de código para el desarrollo del frontend (Angular).
- MySQL Workbench: Herramienta para la administración y diseño de la base de datos.

Opcionales:

- Postman: para hacer pruebas del funcionamiento del back

## Descargar el código fuente

Descargar el código fuente desde el repositorio de GitHub:

https://github.com/celia2mlg/mi-proyecto/

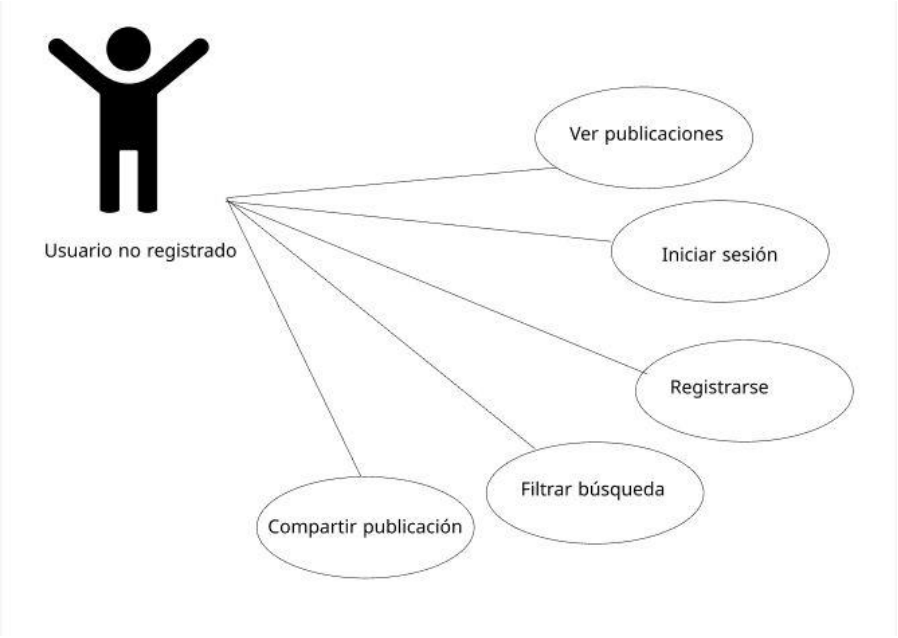
# 3. Análisis

En este apartado podremos ver un análisis de la funcionalidad y la estructura del proyecto con diagramas UML. Los diagramas UML son representaciones gráficas que ayudan a visualizar y documentar el funcionamiento del sistema.

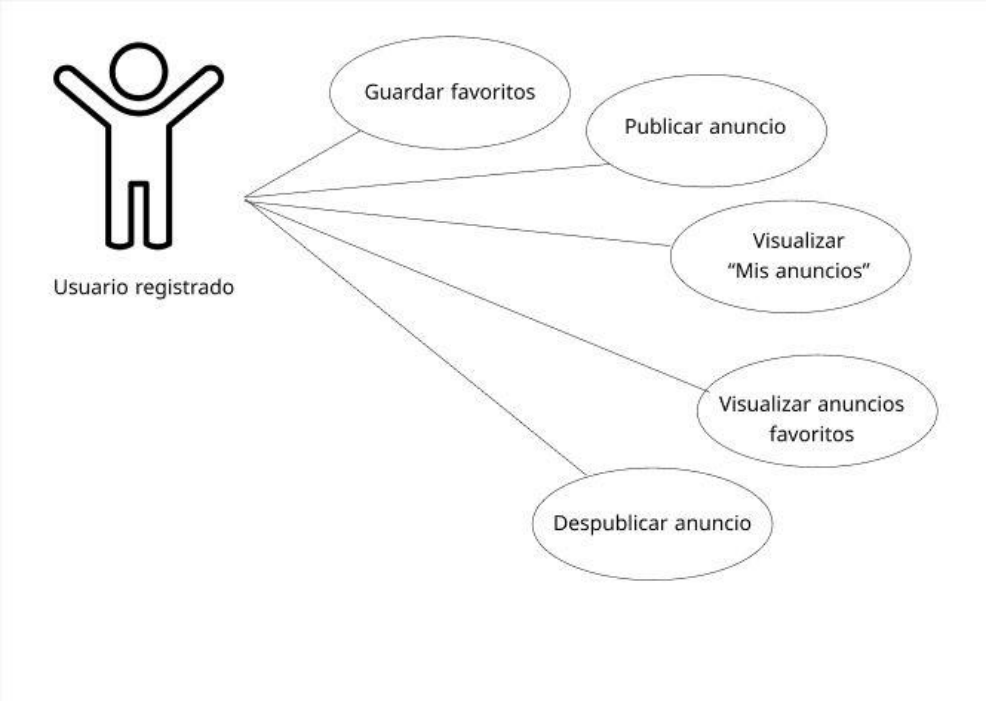
## Casos de uso

Vamos a mostrar a través de los casos de uso como un usuario (actor) interactúa con el sistema, mostrando así las funciones que puede realizar.

Veremos lo que puede hacer un usuario que no está registrado:

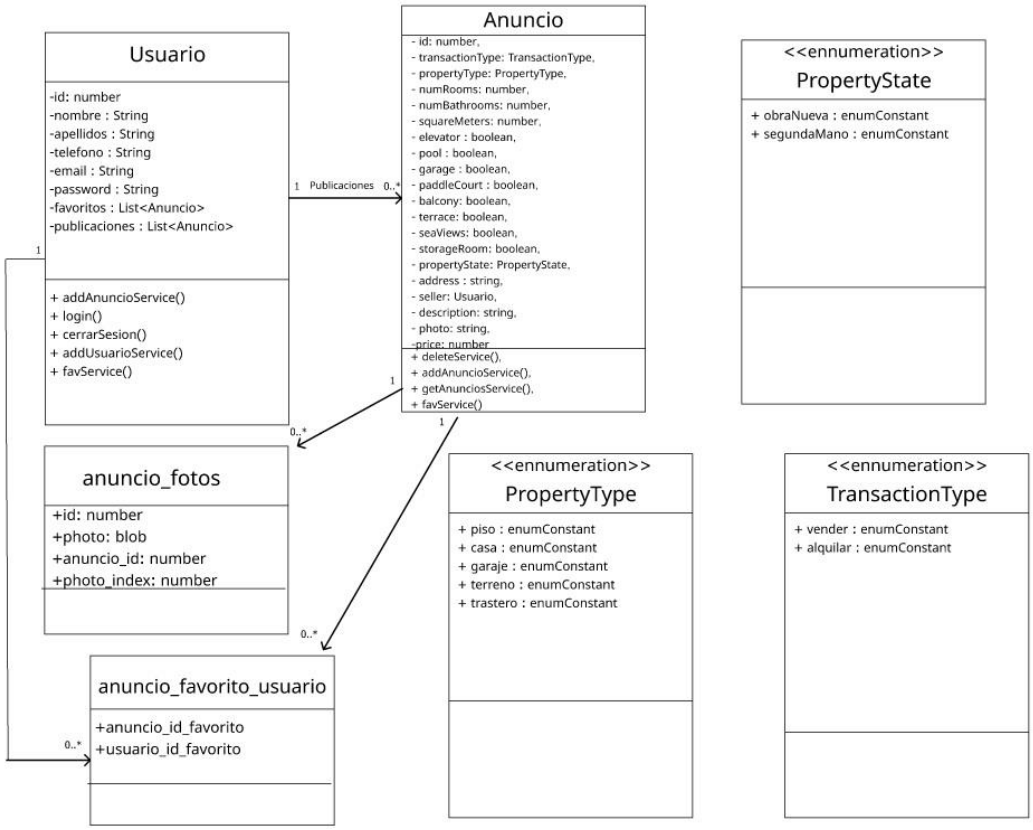


Y aquí veremos lo que puede hacer uno que si esté registrado:



## Diagrama de clases

El diagrama de clases es un tipo de diagrama UML que representa la estructura estática de un sistema, mostrando sus clases, atributos, métodos y las relaciones entre ellas. Es fundamental en el diseño de software orientado a objetos, ya que ayuda a modelar cómo se organizan y comunican los diferentes componentes del sistema.




Aquí explicamos las relaciones entre las diferentes clases que tenemos en el proyecto.

Por un lado, tenemos la relación *OneToMany* entre usuario y anuncio, que representa las publicaciones que puede tener un usuario. Un usuario puede tener ninguno o varios anuncios publicados, mientras que un anuncio puede tener solamente un usuario como vendedor.

Después tenemos otra relación entre estas dos clases que es *ManyToMany*, que representa los favoritos que pueden tener. Un usuario puede tener ninguno o varios anuncios guardados como favoritos, y un anuncio puede tener ninguno o varios usuarios que lo han guardado como favorito. Se crea en la base de datos una tabla intermedia llamada anuncio_favorito_usuario, que está representada en el diagrama.

También tenemos la relación *OneToMany* de anuncio con fotos. Por este motivo se ha creado una tabla a parte en la base de datos.

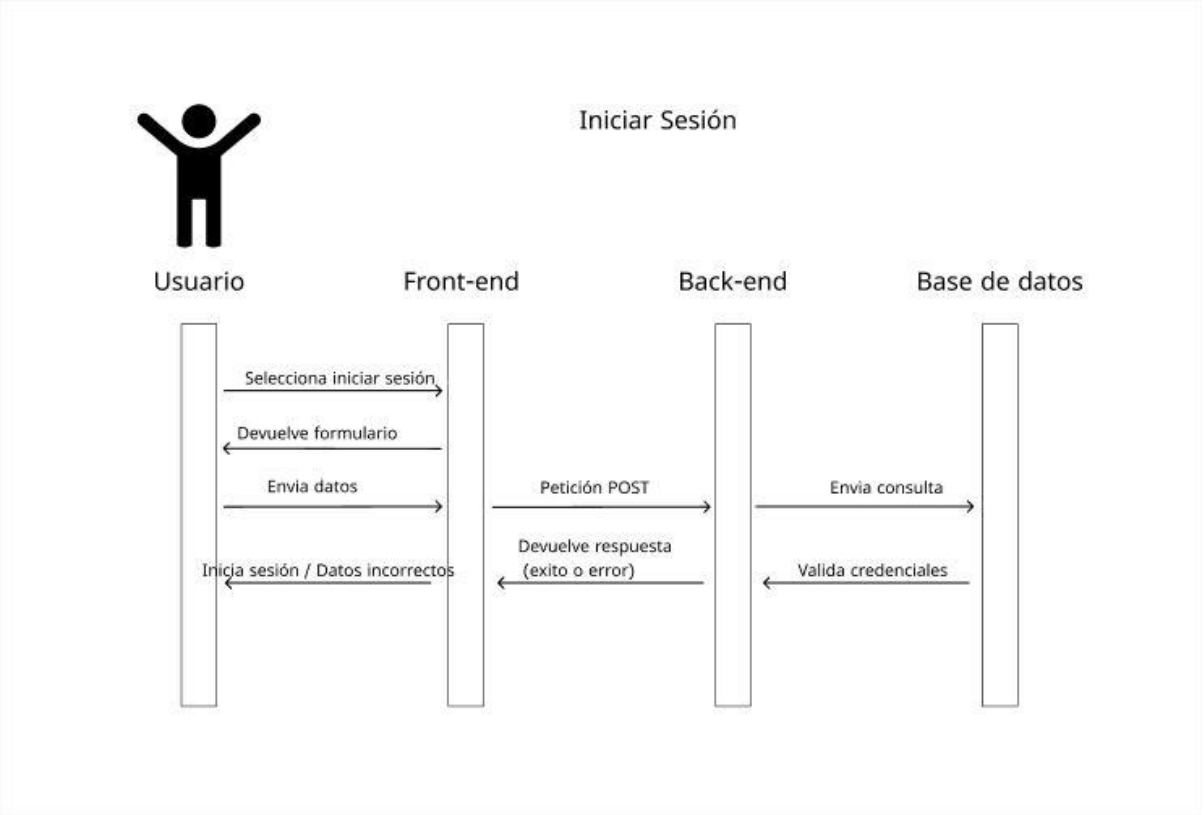
Por último, también vemos representados en el diagrama los *enum*.

## Diagramas de secuencia

Un diagrama de secuencia es una representación visual que muestra, paso a paso, cómo interactúan los distintos actores a través de mensajes o eventos en un proceso. Este tipo de diagrama ilustra el orden y la dependencia temporal de las llamadas o peticiones entre los componentes (front-end, back-end, base de datos), facilitando la comprensión de la lógica de negocio y del flujo de información.

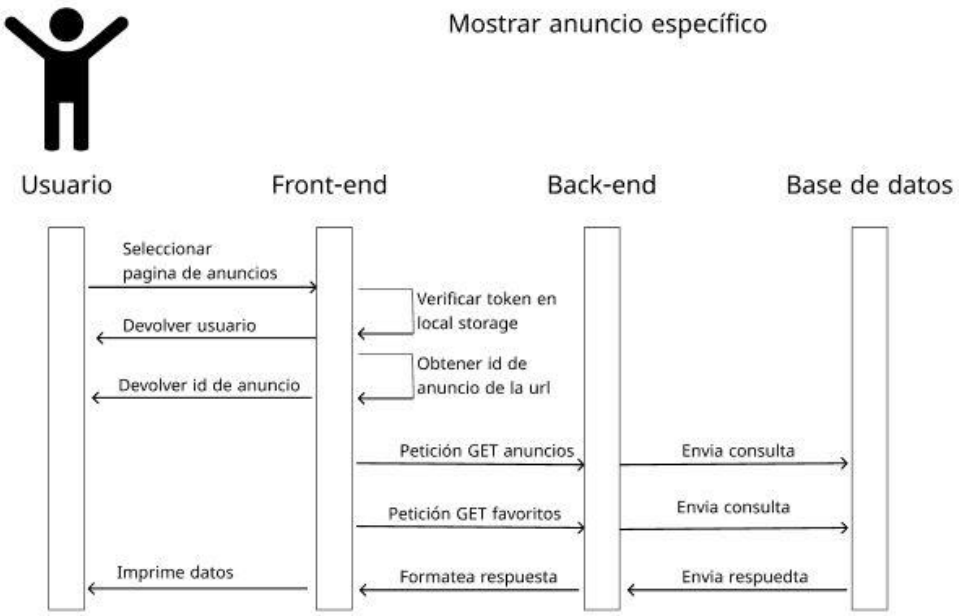
### Diagrama de secuencia Iniciar sesión

Muestra el proceso en el que el usuario, tras pulsar iniciar sesión, introduce sus credenciales (email y password) y se envía la petición a la base de datos para confirmar si son o no correctas.



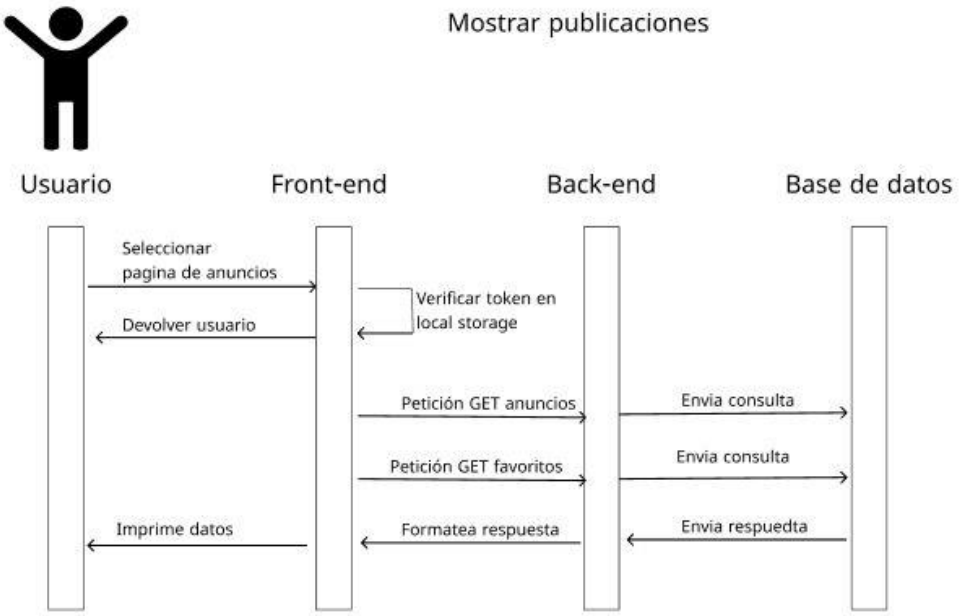
## Diagrama de secuencia Mostrar anuncio específico

Se hace una consulta en primer lugar al LocalStorage, para verificar si hay algún usuario registrado o no, y en caso afirmativo, obtener sus datos. Después se hace una consulta a la URL para comprobar el id del anuncio que necesitamos mostrar. A partir de ahí, se realiza una consulta a la base de datos para obtener los datos de dicho anuncio y si fuese o no favorito del usuario, en caso de haberlo.



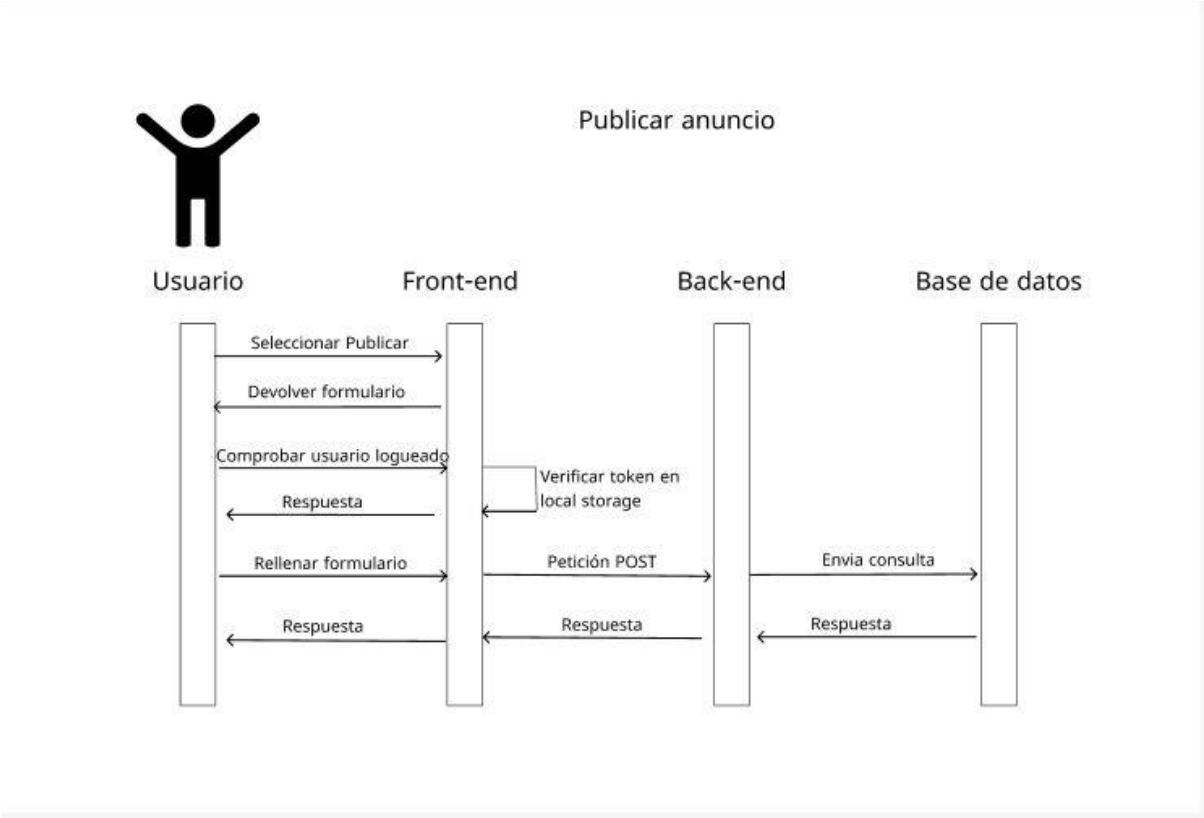
## Diagrama de secuencia Mostrar publicaciones

En primer lugar, se verifica en el LocalStorage el usuario registrado (en caso de haberlo), y después se hace una petición GET a la base de datos para obtener todos los anuncios y los que son favoritos por parte del usuario registrado.



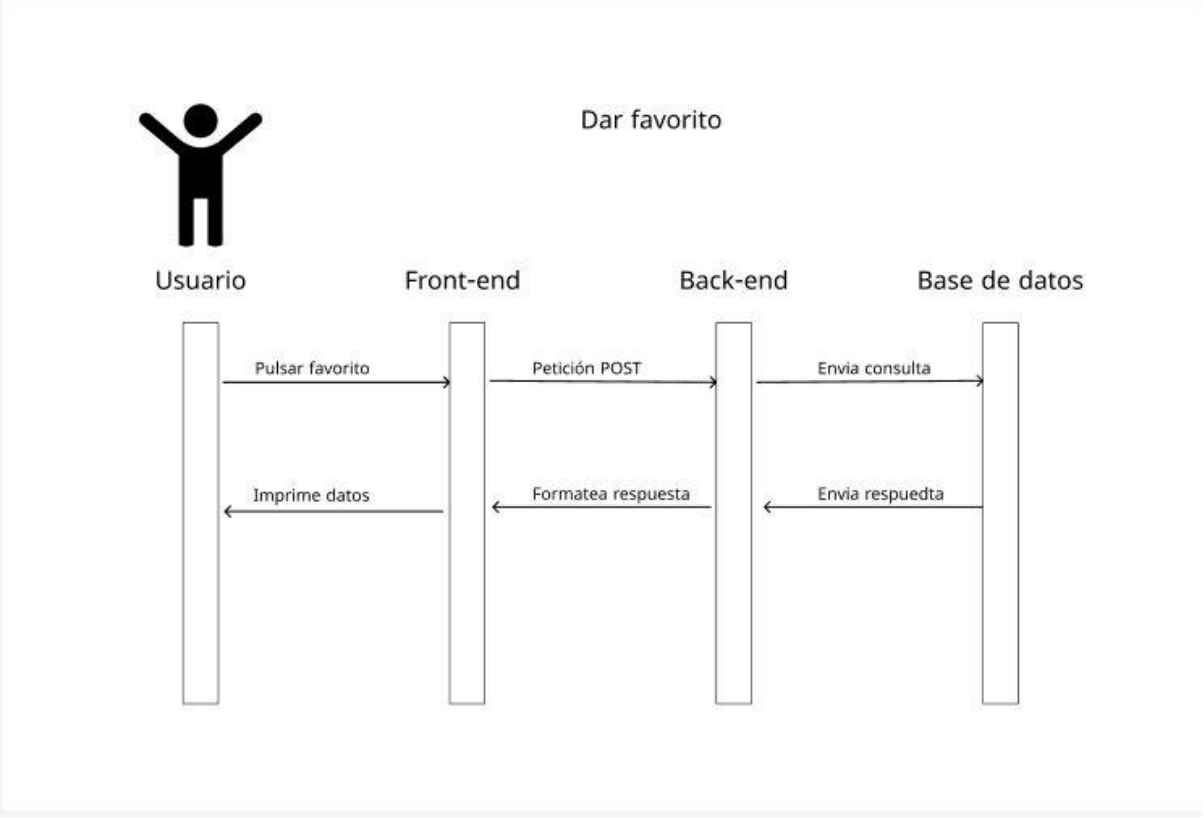
## Diagrama de secuencia Publicar anuncio

Primero se comprueba que haya algún usuario en el LocalStorage, ya que es requisito indispensable para poder hacer una publicación. En caso afirmativo, el usuario rellena el formulario y se envía la petición POST a la base de datos.



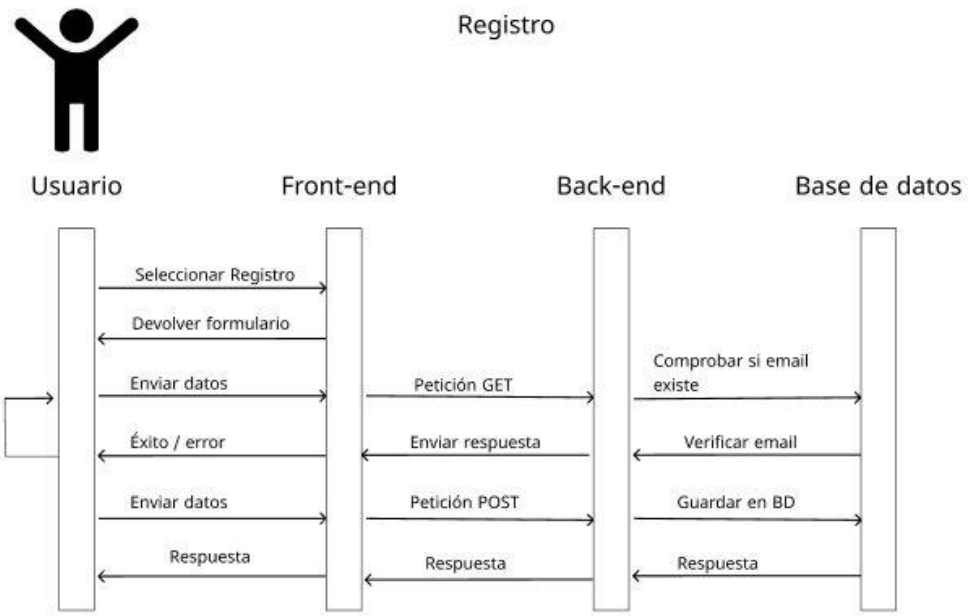
## Diagrama de secuencia Dar favorito

Se realiza al pulsar el botón favorito de las publicaciones. En caso de no ser favorito, se volvería favorito, y viceversa. Después de hacer la modificación del valor de favorito, se envía la petición a la base de datos para actualizarla.



## Diagrama de secuencia Registro

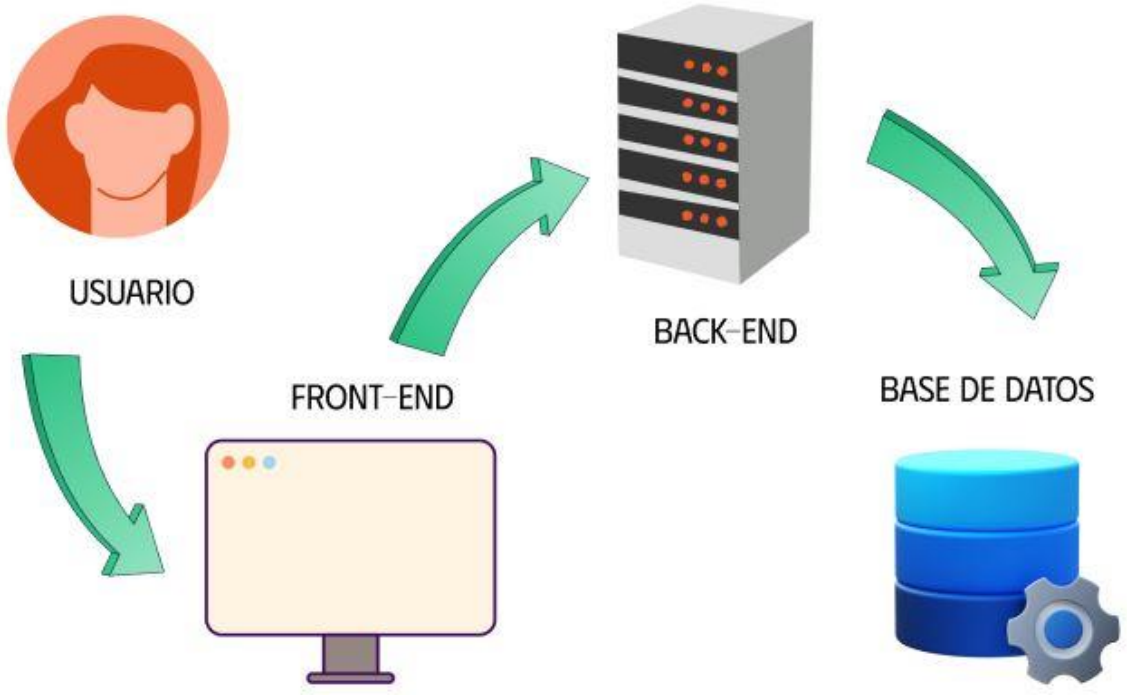
En primer lugar, se selecciona la función de registro. El front-end devolverá un formulario al usuario. Una vez el usuario lo rellene, se emite una petición GET a la base de datos para comprobar que el email sea único. En caso afirmativo, se hará una segunda petición, en este caso POST, para guardar en base de datos el nuevo usuario. En caso negativo, el usuario deberá modificar el valor del email por uno nuevo.



# 4. Diseño

El proyecto está estructurado en tres capas, para poder modificar cada una sin que las otras se vean afectadas, llevando así un orden lógico en la elaboración de este. Se compone así:

- **Capa de presentación**: es la que interactúa directamente con el cliente. Tiene una lógica totalmente funcional y dinámica. Es intuitiva para que el usuario sepa en cada momento dónde puede acudir. Está estructurada de forma que tiene distintos directorios que ordenan su lógica por funcionalidad. Los describiremos más adelante.
- **Capa de lógica**: esta capa interactúa con la capa de presentación y la base de datos, haciendo de intermediaria para un mejor funcionamiento. Se encarga de recibir las solicitudes desde el cliente, y gestionarlas para que lleguen a la base de datos. También está ordenada en directorios en función de su utilidad.
- **Capa de datos**: sirve para almacenar los datos. Es posible tanto modificar, como añadir y eliminar datos. Es la capa de negocio quien interactúa con ella.

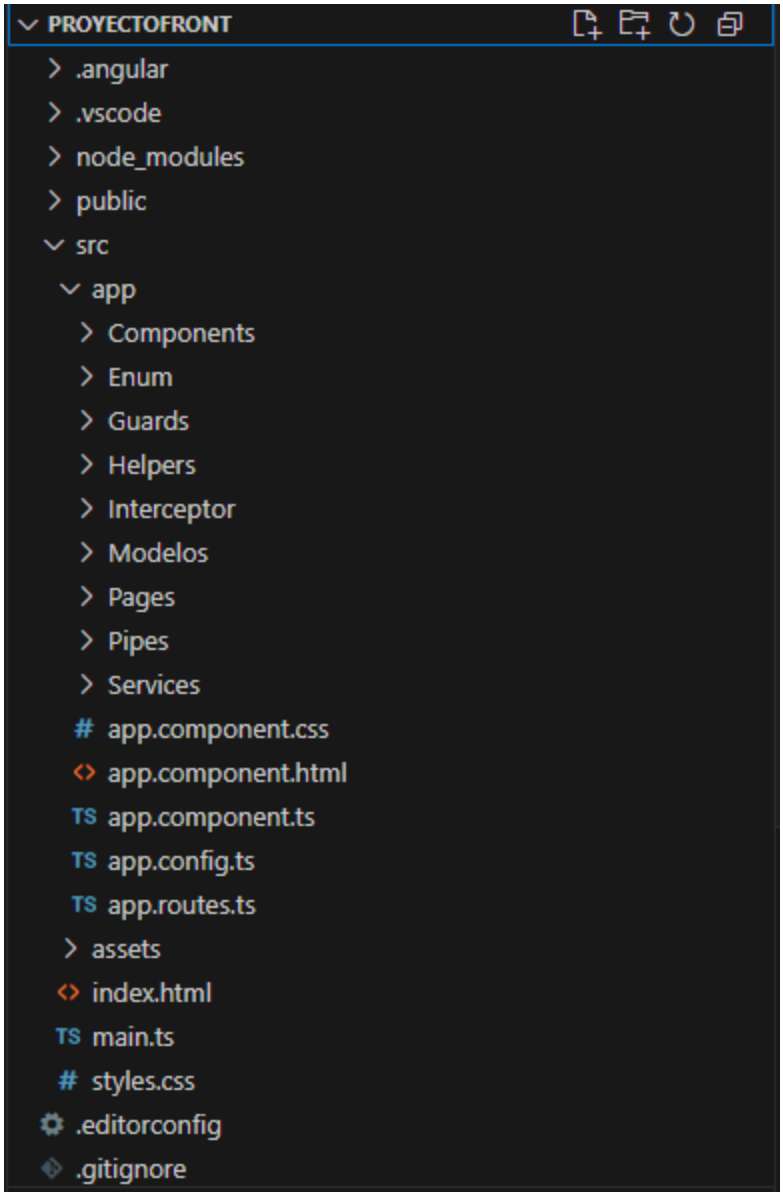


## Capa de presentación

Como he explicado anteriormente, la capa que interactúa con el cliente se ha desarrollado utilizando HTML5, CSS y TypeScript, implementados mediante el framework Angular. Además, se han integrado librerías de Angular Material para optimizar la experiencia visual.

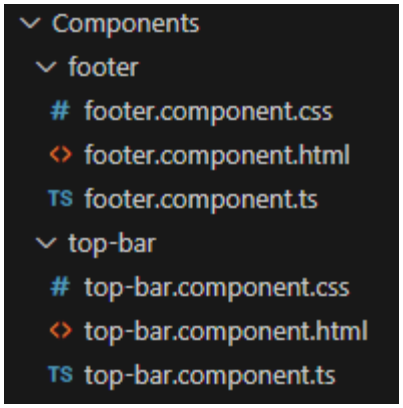



La parte del front de mi proyecto sería la siguiente:



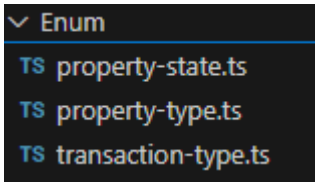

Voy a explicar paso a paso cómo se ha distribuido, para que se entienda el orden que se ha llevado para que sea lo más intuitivo posible.

1. **Componentes**: en la carpeta “componentes” guardo los elementos que serán reutilizados en más de una página (en mi caso en todas). Entre ellos tenemos:



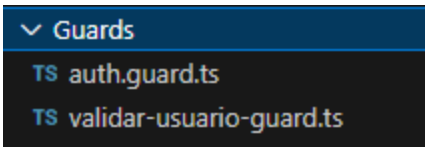

- **Footer**, que es el pie de página. Se verá en todas las paginas del proyecto y en el podremos ver información de la página, así como el mapa interactivo de dónde nos encontramos. Al clicar, nos llevará a Google Maps. También tenemos la información de contacto, con enlace directo a Whatsapp y al teléfono.
- **TopBar**, es la barra superior en la que tenemos el logo y el nombre de la página, que al clicarla nos lleva a la página "home". También tenemos un enlace directo a "inicio sesión" y un menú desplegable a la derecha.

2. **Enum**: tenemos las clases de tipo enumeradas. Tenemos concretamente tres:




- **Property State:** que indica el estado de la propiedad. Puede ser "nueva" o de "segunda mano".
- **Property Type**: indica el tipo de propiedad. Puede ser "piso", "casa", "trastero", "garaje" o "terreno".
- **Transaction Type**: indica el tipo de transacción, puede ser "venta" o "alquiler".

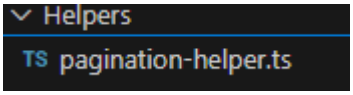
3. **Guards**: tenemos dos Guards, que sirven para proteger rutas y controlar el acceso a ellas. Son los siguientes:



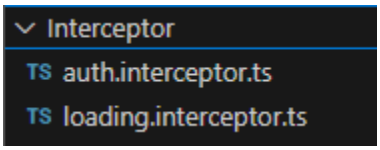

- **Auth guard:** sirve para limitar el acceso a ciertas rutas en caso de no estar logueado. Limita por ejemplo la página "Publicar anuncio" o "Mi perfil".
- **Validar usuario guard**: esta es un poco diferente, sirve para que un usuario no pueda acceder a los datos de otro. Se encarga de comprobar cuál es el usuari1o logueado y compararlo con el id de usuario que se envia en la url

para acceder a su página concreta. Lo empleamos para acceder a la página "Mi perfil" y que no pueda acceder un usuario al perfil de otro.

4. **Helpers**: su utilidad es una ayuda a la paginación de los anuncios publicados. Este nos da opción a seleccionar cuántos anuncios queremos ver por página, y pasar a la siguiente o anterior página.

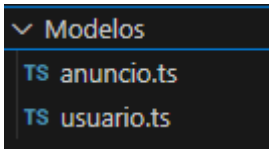



5. **Interceptor**: los interceptores son un tipo de middleware que se utilizan para interceptar las peticiones http y dar la posibilidad de modificar las solicitudes y respuestas antes de que lleguen a su destino o vuelvan al componente que las solicitó. En el proyecto tengo dos:



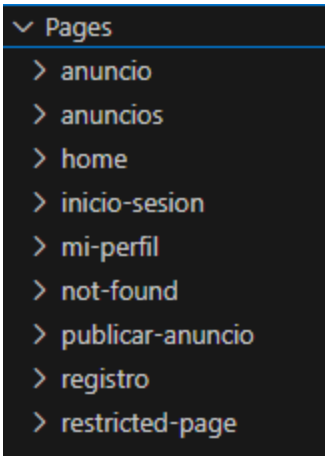

   - **Auth interceptor**: este interceptor obtiene el token del usuario y, si existe, clona la solicitud agregando el token en el header Authorization con el formato Bearer {token}. Esto permite que el backend, que está protegido, valide la autenticación del usuario.
   - **Loading interceptor**: este interceptor inicia un loader justo antes de enviar la solicitud y lo detiene al finalizar la operación, ya sea que la respuesta se haya completado correctamente o se haya producido un error. Esto proporciona feedback visual al usuario sobre el estado de la petición.

6. **Modelos**: son representaciones estructuradas de las entidades del proyecto. En ellas podemos ver las propiedades de cada una de ellas. Sirven para comunicar el front-end con el back-end, ayudando a estructurar la información que se intercambia. Permite estandarizar la forma en la que se organizan estos objetos. En el proyecto tenemos dos:




   - **Anuncio**: tiene propiedades como dirección, precio, número de baños y habitaciones, y algunos boleanos como ascensor, vistas al mar, garaje, etc…
   - **Usuario**: tiene las propiedades de nombre, apellidos, email, teléfono y password.

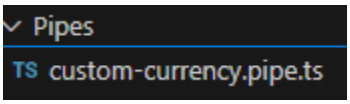
7. **Pages**: son las diferentes vistas de la página. Cada una está relacionada con una ruta de la aplicación. Son los componentes principales que estructuran la navegación y la experiencia del usuario dentro de la plataforma.



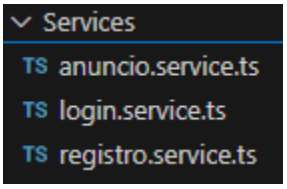

- **Anuncio**: es la vista en la que se ven anuncios específicos, una vez clicado para ver todo el detalle del mismo. En ella podemos ver el precio, todas las imágenes ampliadas, la descripción completa y todos sus datos, como dirección, habitaciones, baños, etc..
- **Anuncios**: es la lista de todos los anuncios que hay publicados. A su izquierda se ve una columna de filtros para poder seleccionar características imprescindibles para tu búsqueda. Aunque presenta una información resumida de cada anuncio, presta funciones muy útiles como dar favorito o contactar con el vendedor.
- **Home**: es la página de inicio, en la que también podemos ver un resumen de lo que podremos hacer en la página, acompañado de fotografias relacionadas con el sector inmobiliario. Desde esa página podemos acceder a un pequeño filtrado de anuncios, a publicar un anuncio y a iniciar sesión o registrarse.
- **Inicio sesión:** esta es la página en la que podremos iniciar sesión. Comprueba la existencia de un email con dicha contraseña y en caso de ser correcta, guarda el token en el local storage para mantener la sesión iniciada. Al loguearse redirige al home page.
- **Mi perfil**: en esta página el usuario podrá acceder a sus datos personales, como nombre completo, email y teléfono. También podrá ver los anuncios que tiene publicados y eliminarlos y ver los anuncios que tiene guardados en favorito. Solo podrá acceder a la página “mi perfil” el usuario registrado, es decir, no se podrá acceder al perfil de otro usuario.
- **Not found**: pagina no encontrada. Se verá en caso de que la página que intenta acceder el usuario no exista.
- **Publicar anuncio**: en esta página, el usuario podrá configurar una propiedad para publicarla. Podrá elegir entre todas las características del inmueble, como tipo de propiedad, si es para vender o para alquilar, dirección, adjuntar fotografías, tamaño, etc.. Al darle a publicar, se subirá a la base de datos a través de una solicitud http POST.

- **Registro**: aquí el usuario podrá registrar una nueva cuenta. La cuenta será válida en caso de que el email no exista previamente. El resto de datos si se pueden repetir. La contraseña deberá ser fuerte, es decir, tener un mínimo de caracteres, alguna mayúscula, minúscula, número y carácter especial. Habrá que introducirla dos veces para asegurarse de que está bien escrita. También se solicitará el número de teléfono y el nombre completo. Estos datos también serán almacenados en una tabla en nuestra base de datos, a través de una solicitud http POST.
- **Restricted page**: esta página sirve para denegar el acceso a ciertos usuarios. La utilizo a la hora de restringir el acceso a la página "mi perfil" en caso de que un usuario quiera acceder a la página "mi perfil" de otro. Se realiza a través del "Validar usuario guard".

8. **Pipes**: sirve para transformar datos en la plantilla de forma reutilizable. En mi caso, lo utilizo para transformar valores númericos en monedas.




9. **Services**: lo utilizamos para reutilizar código que me pueda hacer falta en varias ocasiones, para optimizar el proyecto y evitar duplicaciones innecesarias. También lo utilizo para mediar entre las peticiones http que hace el usuario y las respuestas del back, permitiendo una mejor organización. Tenemos tres servicios.




- **Anuncio service:** lo utilizamos para las funciones necesarias relacionadas con los anuncios.
- **Login service**: lo utilizamos para las funciones necesarias para el inicio de sesion.
- **Registro service**: lo utilizamos para las funciones necesarias para el registro de un nuevo usuario.

## Capa Lógica

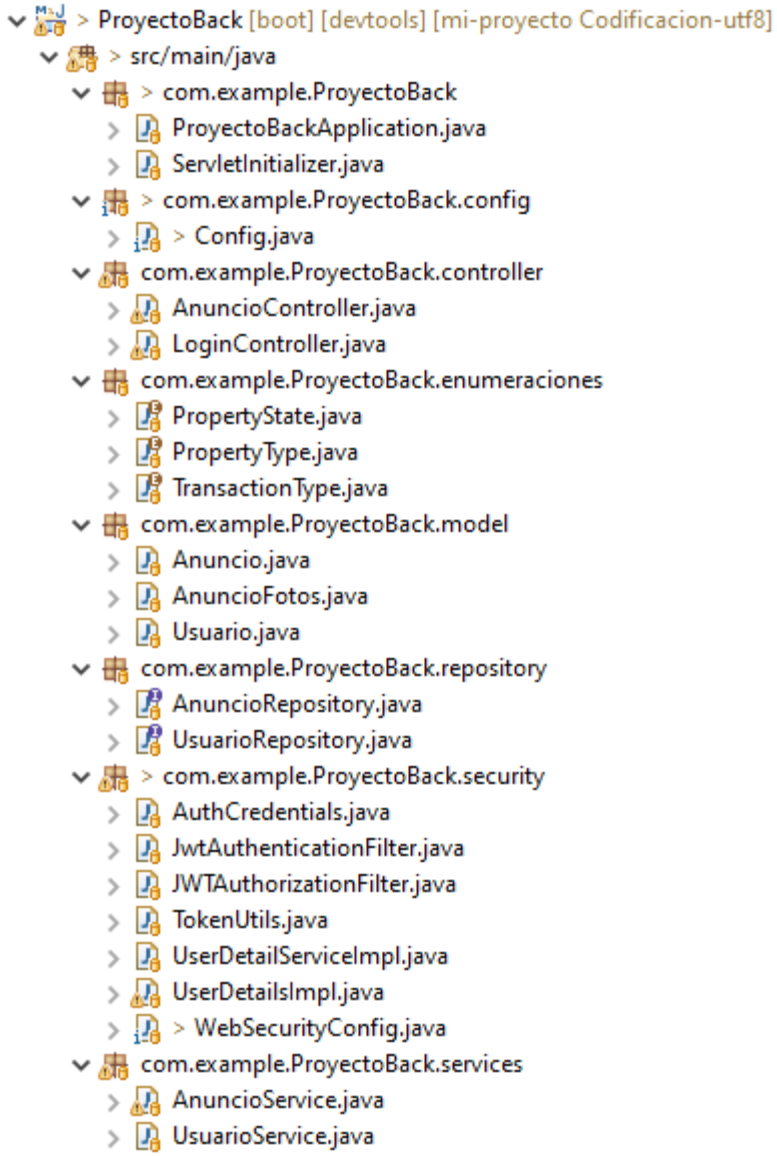
La capa lógica, se encarga de enlazar la interfaz de usuario con la base de datos. Esta capa recibe las peticiones del cliente, y recibe la respuesta de la base de datos, para devolverlas al cliente.
En el proyecto, esta capa está elaborada con el lenguaje Java, a través del framework Spring Boot.







A través de este enlace podemos ver un resumen del back.
http://localhost:8081/swagger-ui/index.html#/




1. **Paquete com.example.ProyectoBack**
   - **ProyectoBackApplication:** Es la clase principal de la aplicación Spring Boot. Contiene el método main() con la anotación *@SpringBootApplication*, que inicializa todo el contexto de Spring y arranca el proyecto.
   - **ServletInitializer**: Facilita el despliegue de la aplicación como un archivo .war en servidores externos (Tomcat, WildFly, etc.). Extiende *SpringBootServletInitializer* y sobreescribe el método *configure*(), necesario para empaquetar el proyecto como WAR en lugar de un JAR ejecutable.
2. **Paquete com.example.ProyectoBack.config**:
   - **Config:** Esta clase implementa WebMvcConfigurer y se encarga de definir la configuración global de CORS para la aplicación. A través del método addCorsMappings se especifica que todas las rutas ("/**") permiten

peticiones desde "http://localhost:4200" utilizando los métodos GET, POST, PUT, DELETE y OPTIONS, permitiendo todos los headers, exponiendo todos los headers en las respuestas y habilitando el envío de credenciales.

3. **Paquete com.example.ProyectoBack.controller**
   - **AnuncioController**: Controlador que gestiona las peticiones relacionadas con la entidad Anuncio (creación, obtener todos los anuncios, obtener anuncio concreto, marcar como favorito, etc.). Expone endpoints REST usando anotaciones como @*RestController* y @*RequestMapping*. Está mapeado con la ruta /anuncio.
   - **LoginController**: Maneja las peticiones relacionadas con los usuarios, como el registro, el encontrar usuario, la obtención de todos los usuarios, etc.. Expone endpoints REST usando anotaciones como @*RestController* y @*RequestMapping*. Está mapeado con la ruta /login.

4. **Paquete com.example.ProyectoBack.enumeraciones :** Enumeraciones que definen valores fijos para ciertas propiedades de los anuncios.
   - **PropertyState:** "nueva" o "segunda mano".
   - **PropertyType:** "piso", "casa", "trastero", "garaje" o "terreno".
   - **TransactionType:** "venta" o "alquiler".

5. **Paquete com.example.ProyectoBack.model :** En este paquete guardamos los modelos, es decir las entidades con anotación @*Entity ,* que representan las tablas de la base de datos.
   - **Anuncio**: Entidad que representa un anuncio o publicación inmobiliaria. Contiene campos como dirección, descripción, precio, tipo de propiedad, etc. Lleva anotaciones JPA para mapearse a la base de datos.
   - **AnuncioFotos**: Entidad que gestiona las imágenes asociadas a un anuncio. Se crea una entidad aparte debido a la relación "*OneToMany*" de anuncio con fotos, pudiendo tener sus propias propiedades.
   - **Usuario**: Entidad que representa a un usuario dentro del sistema (email, nombre, apellidos, teléfono, etc.). Se relaciona con la base de datos para almacenar y consultar la información de los usuarios. Lleva anotaciones JPA para mapearse a la base de datos.

6. **Paquete com.example.ProyectoBack.repository**
   - **AnuncioRepository:** Interfaz que extiende JpaRepository. Hereda automáticamente un conjunto de métodos para gestionar operaciones CRUD sobre la entidad Anuncio. Contiene métodos para buscar, guardar o eliminar anuncios en la base de datos sin necesidad de implementar la lógica explícitamente.

- **UsuarioRepository**: Interfaz que extiende JpaRepository para gestionar operaciones CRUD sobre la entidad Usuario. Puede incluir consultas personalizadas, como búsqueda por email o nombre.

7. **Paquete com.example.ProyectoBack.security**
   - **AuthCredentials**: Esta clase es un POJO (Plain Old Java Object) que contiene dos atributos: *email* y *password*. Se utiliza para mapear los datos que se envían en el cuerpo de la petición de autenticación (generalmente en formato JSON) y se usa en el filtro de autenticación para extraer las credenciales del usuario.
   - **JwtAuthenticationFilter:** Extiende de *UsernamePasswordAuthenticationFilter* y se encarga de gestionar el proceso de autenticación. En el método *attemptAuthentication*, lee el cuerpo de la petición para convertirlo en un objeto *AuthCredentials* y crea un *UsernamePasswordAuthenticationToken* para que el gestor de autenticación lo valide. Si la autenticación es exitosa, en el método *successfulAuthentication* se genera un token JWT mediante la clase *TokenUtils*, se añade al encabezado de la respuesta y se envía de vuelta en el cuerpo de la respuesta.
   - **JWTAuthorizationFilter**: Esta clase, que extiende de *OncePerRequestFilter*, actúa en cada petición para comprobar si en el encabezado se incluye un token JWT válido. Si detecta un token que comienza con "Bearer ", lo extrae y utiliza *TokenUtils* para obtener el objeto de autenticación. Si el token es válido, se establece la autenticación en el contexto de seguridad de Spring, permitiendo el acceso a recursos protegidos.
   - **TokenUtils:** Es una clase de utilidad que maneja la creación y validación de tokens JWT. En *createToken*, genera un token que incluye datos adicionales (por ejemplo, el nombre del usuario) y establece una fecha de expiración. En *getAuthentication*, valida el token recibido y, de ser correcto, extrae el email del usuario para crear un objeto *UsernamePasswordAuthenticationToken* que luego se utiliza para establecer la identidad del usuario en el sistema.
   - **UserDetailsImpl**: Esta clase implementa la interfaz *UserDetails* y se encarga de adaptar el modelo *Usuario* al formato que requiere Spring Security. Proporciona los métodos necesarios para obtener el email, la contraseña y otros atributos del usuario, así como indicadores sobre el estado de la cuenta (no expirada, no bloqueada, etc.). Además, incluye un método extra para obtener el nombre del usuario.
   - **UserDetailsServiceImpl**: Implementa la interfaz *UserDetailsService* de Spring Security. Su función es cargar los detalles de un usuario a partir del email, utilizando el *UsuarioRepository*. Si el usuario existe, se devuelve un

objeto *UserDetailsImpl* que encapsula la información del usuario; de lo contrario, se lanza una excepción indicando que el usuario no existe.

- **WebSecurityConfig:** Esta clase configura la seguridad de la aplicación utilizando Spring Security. Define la cadena de filtros (*SecurityFilterChain*) que incluye filtros personalizados para la autenticación y autorización basados en JWT, y configura el *AuthenticationManager* con un *UserDetailsService* y *BCryptPasswordEncoder* para la gestión de contraseñas. Además, registra un filtro de codificación (*CharacterEncodingFilter*) para asegurar que las respuestas se envíen en UTF-8. La configuración de CORS se activa mediante *cors(withDefaults()*), delegando la configuración global al bean definido en Spring MVC. Define las reglas de acceso a los distintos endpoints (indicando qué rutas son públicas y cuáles requieren autenticación).

8. **Paquete com.example.ProyectoBack.services**
   - **AnuncioService:** Contiene la lógica de negocio relacionada con los anuncios. Métodos para crear, obtener anuncios,etc.. Llama a *AnuncioRepository* y encapsula validaciones.
   - **UsuarioService:** Contiene la lógica de negocio relacionada con los usuarios. Métodos para registrar nuevos usuarios, obtener mis anuncios, etc. Llama a *UsuarioRepository* y se integra con la capa de seguridad para validar las credenciales de usuario.

## Capa de datos

En esta capa es en la que se almacenan los datos. Se encarga de gestionar la persistencia y el acceso a la información. En ella se definen las entidades del dominio, las cuales se mapean a la base de datos mediante un ORM (JPA).

Esta capa incluye repositorios y servicios que proporcionan métodos CRUD y consultas específicas, garantizando la integridad, consistencia y seguridad de los datos. Así, las capas superiores (lógica de negocio y presentación) interactúan de forma desacoplada con la base de datos, lo que favorece la escalabilidad y el mantenimiento del sistema.

Se ha usado como sistema de gestión de base de datos **Maria DB**, junto con **MySQL Workbench** para diseñar y administrar los esquemas.

En el proyecto existen 4 tablas.

### Tabla Anuncio

En esta tabla se almacenan los anuncios que se publican. Esta tabla almacena los siguientes datos:



| id | address | balcony | elevator | garage | num_bath | num_room | paddle_ | photo | pool | property_state | property_type | sea_vi | square_mete | storage_ro | terrace | transactio | seller_id | description | price |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Calle Oregon, Los Cristianos, España | 1 | 0 | 0 | 5 | 45 | 1 | NULL | 1 | secondHand | Trastero | 1 | 5 | 0 | 0 | sell | NULL | Piso muy bonito en Malaga | 129000 |
| 2 | calle lario | 0 | 1 | 0 | 10 | 10 | 0 | NULL | 0 | secondHand | Trastero | 0 | 10 | 0 | 1 | rent | NULL | Piso muy bonito en Malaga | 145000 |
| 3 | Calle Larios, Málaga, España | 0 | 1 | 0 | 10 | 10 | 0 | NULL | 0 | secondHand | Trastero | 0 | 10 | 0 | 1 | rent | NULL | Piso muy bonito en Malaga | 150000 |
| 4 | Calle Larios, Málaga, España | 0 | 1 | 1 | 2 | 4 | 0 | NULL | 1 | newBuild | Piso | 1 | 120 | 0 | 1 | sell | 1 | Piso muy bonito en Malaga | 158000 |
| 5 | Calle Oregano, Benalmádena, España | 0 | 0 | 1 | 2 | 4 | 0 | NULL | 1 | secondHand | Terreno | 0 | 120 | 0 | 0 | sell | 1 | Piso muy bonito en Malaga | 160990 |
| 6 | Calle Prensa, Málaga, España | 0 | 1 | 0 | 5 | 5 | 0 | NULL | 0 | secondHand | Garaje | 0 | 5 | 0 | 1 | rent | 1 | Piso muy bonito en Malaga | 185000 |
| 7 | Avenue de Wagram, París, Francia | 0 | 0 | 0 | 1 | 1 | 0 | NULL | 1 | secondHand | Casa | 0 | 120 | 0 | 1 | sell | 1 | Piso muy bonito en Malaga | 240000 |
| 8 | Bulevar Louis Pasteur, Málaga, España | 0 | 1 | 0 | 2 | 4 | 1 | NULL | 0 | newBuild | Casa | 0 | 120 | 0 | 1 | sell | 1 | Piso muy bonito en Malaga | 200000 |
| 9 | Calle Oregano, Benalmádena, España | 1 | 1 | 1 | 1 | 2 | 1 | NULL | 1 | newBuild | Casa | 0 | 85 | 0 | 1 | sell | 1 | Piso muy bonito en Malaga | 120000 |
| 11 | Calle Trinidad Grund, Málaga, España | 1 | 0 | 1 | 2 | 8 | 1 | NULL | 1 | newBuild | Casa | 0 | 200 | 0 | 0 | rent | 1 | Piso muy bonito en Malaga | 140000 |
| 12 | Calle Carretería, Málaga, España | 0 | 1 | 0 | 2 | 5 | 0 | NULL | 0 | secondHand | Casa | 0 | 200 | 0 | 1 | sell | 1 | Piso muy bonito en Malaga | 160000 |
| 13 | Calle Oregon, Los Cristianos, España | 0 | 1 | 0 | 2 | 4 | 0 | NULL | 0 | secondHand | Casa | 0 | 65 | 0 | 1 | rent | 2 | Casa preciosa en la costa... | 125000 |
| 14 | Calle José Sánchez Sáez, Elche, España | 0 | 1 | 0 | 1 | 3 | 0 | NULL | 0 | secondHand | Casa | 0 | 92 | 0 | 0 | sell | 2 | Vivienda ubicada cerca de... | 135000 |
| 15 | Calle José Sánchez Sáez, Elche, España | 0 | 1 | 0 | 1 | 3 | 0 | NULL | 0 | secondHand | Casa | 0 | 92 | 0 | 0 | sell | 1 | Vivienda ubicada cerca de... | 135000 |
| 16 | Calle José Sánchez Sáez, Elche, España | 0 | 1 | 0 | 1 | 3 | 0 | NULL | 0 | secondHand | Casa | 0 | 92 | 0 | 0 | sell | 1 | Vivienda ubicada cerca de... | 135000 |
| 17 | Calle José Sánchez Sáez, Elche, España | 0 | 1 | 0 | 1 | 3 | 0 | NULL | 0 | secondHand | Casa | 0 | 92 | 0 | 0 | sell | 2 | Vivienda ubicada cerca de... | 135000 |
| 18 | Calle Juan de la Cierva, Marbella, Esp... | 0 | 0 | 1 | 2 | 5 | 1 | NULL | 0 | secondHand | Casa | 0 | 120 | 1 | 1 | sell | 1 | Preciosa casa en marbella | 130000 |
| 19 | Calle Salamanca, Móstoles, España | 0 | 1 | 0 | 2 | 4 | 1 | NULL | 0 | newBuild | Casa | 0 | 120 | 0 | 1 | rent | 2 | Precioso piso en Madrid, c... | 190000 |
| 20 | Calle la Tierra, España | 1 | 1 | 0 | 86 | 786 | 0 | NULL | 0 | secondHand | Garaje | 1 | 86 | 0 | 0 | rent | 2 | jkasdhaskldhasokljasklj | 63232... |
| 21 | Calle Móstoles, Gijón, España | 1 | 1 | 1 | 45 | 45 | 1 | NULL | 1 | secondHand | Terreno | 1 | 45 | 1 | 0 | rent | 2 | jkasdjkashaslkjhadsk, | 580000 |

anuncio 1 ×  Apply  Revert

- *Id*: este campo está definido con la propiedad "*Auto-Increment*" de modo que cada vez que se inserta un nuevo registro, la base de datos asigna automáticamente un valor incremental único a la columna. Esto evita la necesidad de especificar manualmente el identificador y garantiza que no se produzcan duplicados.

- Booleanos: están definidos con 0 en caso de ser *false*, y 1 en caso de ser *true*. El tipo de dato es bit(1). Estos son *balcony, elevator, garage, paddle-court, pool, sea-views, storage-room y terrace*.
- *Num-bathrooms* y *num-rooms:* están definidos por un carácter numérico (int(11)), en el que se indican el número de baños y de habitaciones.
- *Square-meters:* indica los metros cuadrados de la propiedad. También debe ser un carácter numérico (int(11)).
- *Transaction-type:* es un *enum*. Sus valores pueden ser: *sell o rent,* en función de lo que el propietario desee hacer con su propiedad (alquilarla o venderla).
- *Property-type:* es un *enum* también. Sus valores varían en función del tipo de propiedad que sea. Pueden ser *trastero, garaje, piso, casa o terreno*.
- *Property-state:* es otro *enum*. Sus valores varían en función de si es una obra nueva (*newBuild)* o de segunda mano (*secondHand*).
- *Price:* también puede ser únicamente un carácter numérico (double). Indica el precio del inmueble anunciado.
- *Description:* es una variable para que el usuario pueda añadir información adicional de su anuncio. Es un varchar(1000), por lo que tiene un límite de 1000 caracteres.

## Tabla usuarios

Es la tabla que almacena los datos de los usuarios que se van registrando. Los campos que incluyen son:

| id | apellidos | email | nombre | password | telefono |
|---|---|---|---|---|---|
| 1 | Martin-Mora | eva@eva.es | Eva | $2a$10$9XxAGG0SPgd8aSrnAE4oNexVEvE/oNJpQrqGAslDOXBfwl... | 654852963 |
| 2 | Moran | celia@celia.es | Celia | $2a$10$VR2ftg2AdpxAxLDo3.ljmut08sktQoynj/7KMNPTGy6R5W... | 636295971 |
| 3 | Morán Gámez | celia2@celia.es | Celia | $2a$10$x9bXDQjgt1l2Q23p1q4DP.pEUhPmSPDSmJEv0R.LKA8KH... | 636295971 |
| 4 | Flores | pepa@pepa.es | Pepa | $2a$10$iYILQVbepgAGGJ/SKrH4cOQQunCMK//8pVCgManNgGMg... | 636295971 |
| 5 | Pedro | paco@paco.es | Paco | $2a$10$GlWubSuiNF/LWfUMg1tnM.Xc6yFFhREKBbyLv7Aa8em.i9... | 85274189 |
| 6 | Fernandez | fer@fer.es | Fernando | $2a$10$9sCVFXXIeKxZpc.oaF6Mlu6YTMO.URNJfNLB26DQlY2FW... | 7418529 |
| 7 | Cazuela | carmen@cazue... | Carmen | $2a$10$jIOFcvK7kgI0iGIdVT2cU.nC4NlGG7LsskBQM.KnhcCdZcQj... | 56987456 |
| 8 | Cazuela | carmen@sarte... | Carmen | $2a$10$Xn43ABBTn56fFw5TEPme/O2zcEix1FRGDtGaXw3bvTNU1... | 56987456 |
| 9 | celia | celia@celiacelia... | celia | $2a$10$Vyc4HujIRWtpuemnxHE.KucxMWfaK1RXUmAzuf36JOTiV... | 741852963 |
| 10 | Cármencita | carmen@carm... | Cármen | $2a$10$IT880htSCP4zaEHi7xixBOvyh/B.DTzjEz79zVV7OYMgREXl... | 741852963 |
| 11 | Cármencita | carmen2@car... | Carmen | $2a$10$IhjCVf8zKVuNsznEmiVJf.AwT6xkAQ9phkhX056ZVtb8pjA5... | 741852963 |

- *Id:* es un valor generado automáticamente (con auto_increment). Es la primary key. No puede ser nulo y no puede haber dos iguales.
- *Nombre y apellidos:* son del tipo varchar(255).
- *Email:* también es un varchar(255), tiene la peculiaridad de que no puede ser duplicado, es decir, si existe un email asociado a un usuario, no podrá usarse para ningún otro.
- *Telefono:* es del tipo int(11).
- *Password:* se almacena en la base de datos como un varchar(255), pero se almacena codificada, por lo que no será posible ver su valor real desde la base de datos con el fin de proteger a los usuarios.

## Tabla anuncios_favoritos_usuarios

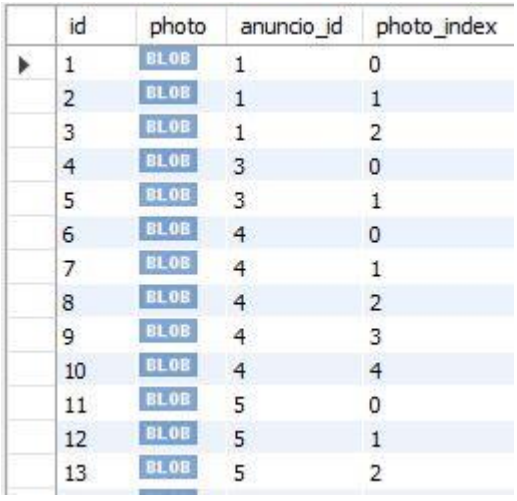
Representa la relación de “favoritos” entre usuarios y anuncios, de modo que un usuario puede marcar varios anuncios como favoritos y un anuncio puede ser favorito de varios usuarios. Es una tabla intermedia. Consta de columnas como anuncio_id_favorito y usuario_id_favorito, que funcionan como llaves foráneas apuntando a las tablas anuncio y usuario respectivamente, reflejando así la relación *ManyToMany*.



| anuncio_id_favorito | usuario_id_favorito |
|---|---|
| 18 | 1 |
| 17 | 2 |
| 13 | 2 |
| 14 | 2 |
| 6 | 2 |
| 11 | 1 |
| 11 | 2 |
| 21 | 2 |
| 1 | 1 |
| 1 | 2 |
| 5 | 1 |
| 25 | 2 |
| 22 | 2 |

## Tabla anuncio_fotos

Guarda las imágenes asociadas a cada anuncio. Tiene un id (clave primaria), un campo BLOB llamado photo (donde se almacena la imagen en binario), un anuncio_id que referencia a la tabla anuncio (indicando a qué propiedad pertenece la imagen) y un photo_index para indicar la posición de cada imagen dentro de ese anuncio.



| id | photo | anuncio_id | photo_index |
|---|---|---|---|
| 1 | BLOB | 1 | 0 |
| 2 | BLOB | 1 | 1 |
| 3 | BLOB | 1 | 2 |
| 4 | BLOB | 3 | 0 |
| 5 | BLOB | 3 | 1 |
| 6 | BLOB | 4 | 0 |
| 7 | BLOB | 4 | 1 |
| 8 | BLOB | 4 | 2 |
| 9 | BLOB | 4 | 3 |
| 10 | BLOB | 4 | 4 |
| 11 | BLOB | 5 | 0 |
| 12 | BLOB | 5 | 1 |
| 13 | BLOB | 5 | 2 |

# 5. Ejemplo de implementación

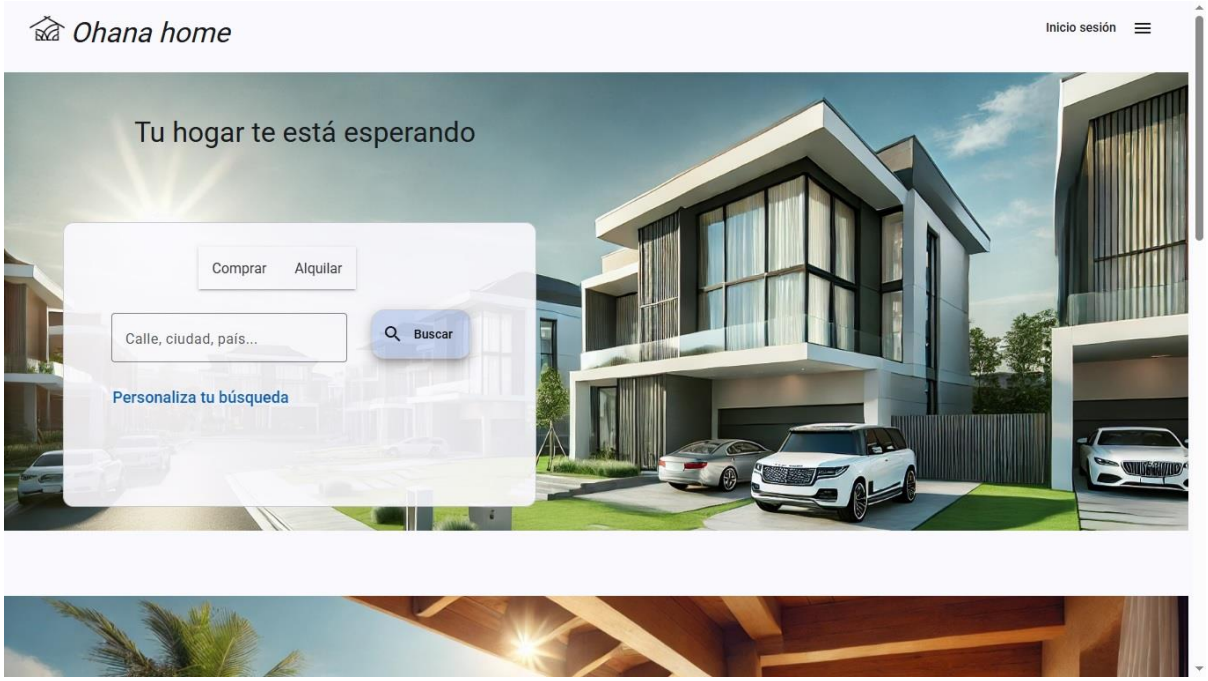
## Parte pública

La parte pública se caracteriza por ser accesible por cualquier usuario, esté o no registrado.

### Home Page

La Home page es la página principal del proyecto, a la que deriva en caso de no acceder a ninguna ruta. En mi proyecto, resume un poco lo que se puede hacer en la página.

Se puede acceder a un pequeño filtro resumido para la búsqueda de anuncios publicados, se puede acceder a publicar un anuncio nuevo o incluso iniciar sesión o registrarse en caso de nuevos usuarios.

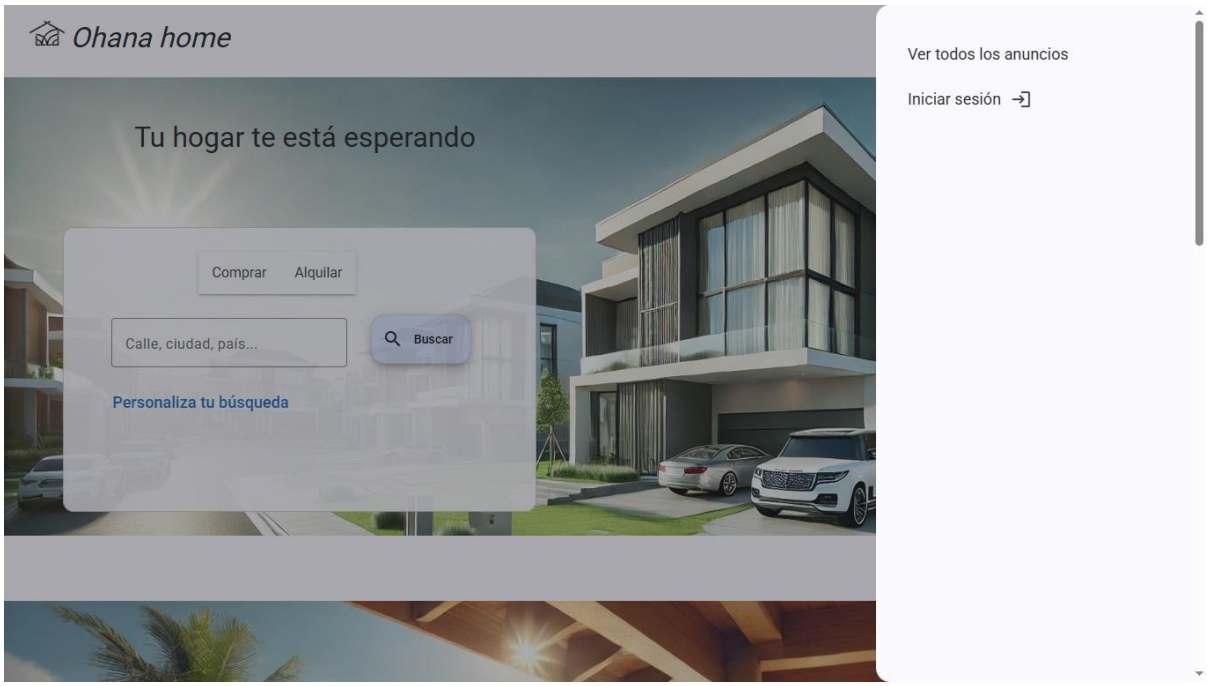



Al comienzo de la página, empezando por arriba, tenemos el componente *Topbar*.

Es común en todas las páginas del proyecto. En él podemos ver el logo y el nombre de la web, en el cual al pulsar accedemos de vuelta a "*Home page*", estemos en la ruta que estemos.

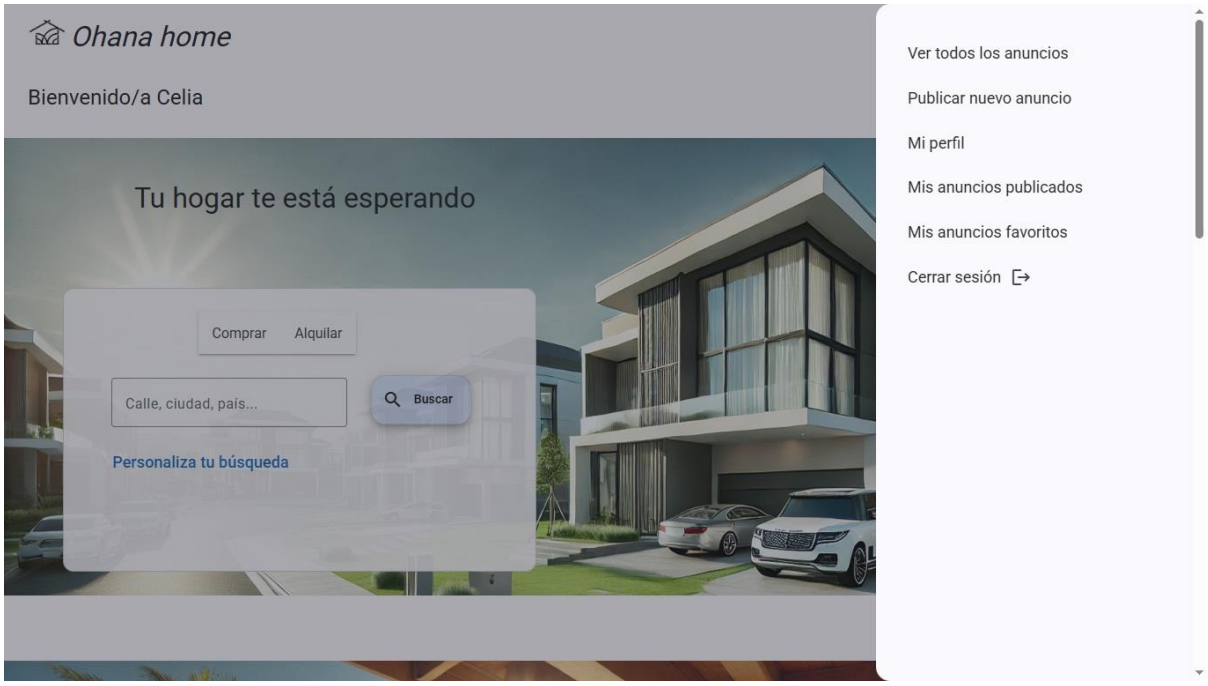
A la derecha del *Topbar* tenemos un acceso directo a "*inicio sesión*", que se ocultará en caso de ya estar la sesión iniciada.

En el extremo de la derecha del *Topbar* tenemos un menú *drawer* que se abre desde la derecha. En caso de tener la sesión cerrada, el *mat-drawer* sería así:




Solo aparecería las opciones públicas, que en este caso son: ver todos los anuncios e iniciar sesión para poder acceder al resto de funcionalidades.

En caso de tener la sesión iniciada, el *mat-drawer* cambiaría:



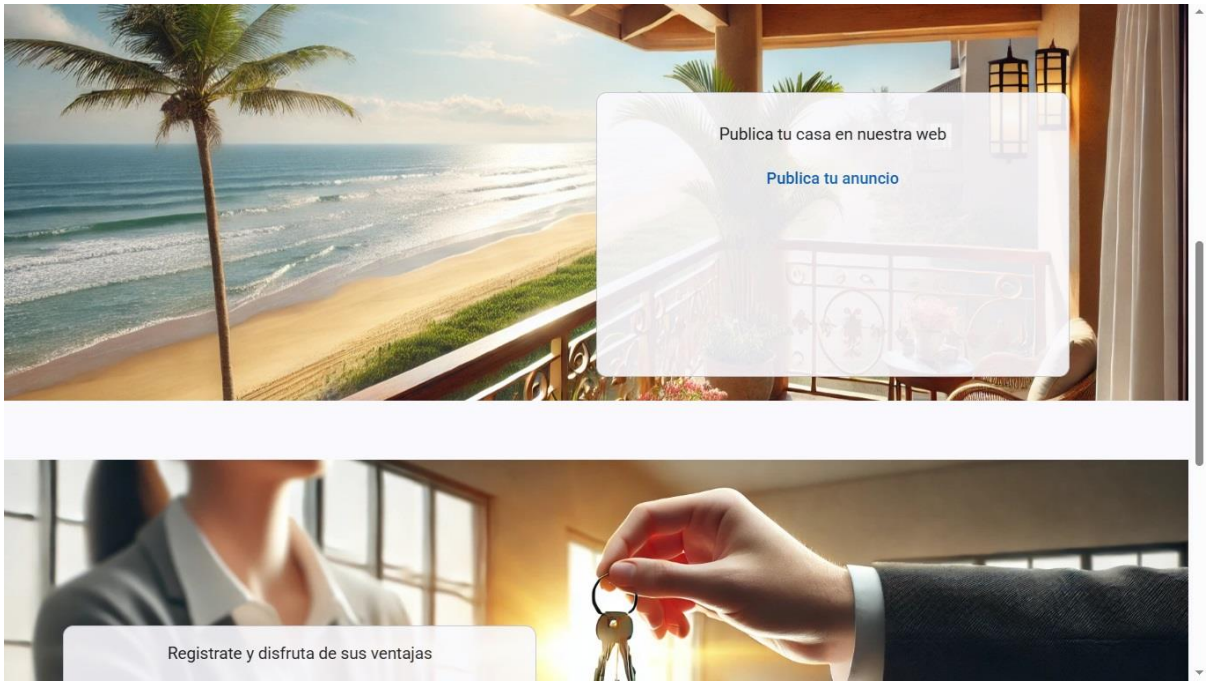

En este caso si tenemos más funciones posibles. Serían: ver todos los anuncios, publicar uno nuevo, acceder al perfil del usuario logueado, filtrar los anuncios y que se vean solo los publicados por el usuario y ver los favoritos de este. También tenemos la opción de cerrar sesión, eliminando el token del localhost.

También aparece una señal para identificar que el inicio de sesión fue correcto, *"Bienvenido/a nombreUsuario"*. De esta forma, sabremos que el inicio fue exitoso y que estamos logueados.

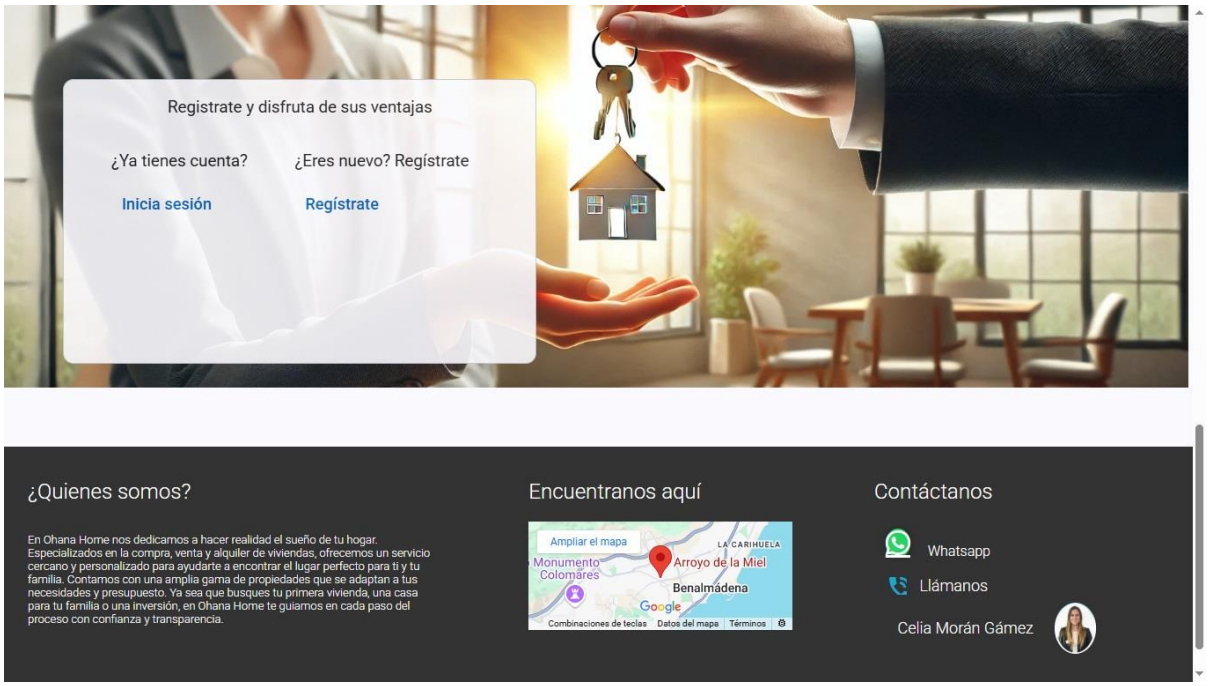
Lo primero que se ve en la página de inicio es la pequeña opción de filtrado, en la que se puede elegir entre si quiere comprar o alquilar, puede introducir una dirección para buscar calle, ciudad o país, y si quisiera más filtros le deriva a la página de anuncios en la que viene la columna de filtros completa.

En caso de darle a buscar sin filtros aplicados, mostraría todos los anuncios.

A continuación, la página de inicio en caso de hacer scroll hacia abajo:




Mostramos la opción de publicar un nuevo anuncio (solo disponible para usuarios registrados). Más abajo encontraremos los enlaces con la página tanto de inicio de sesión como de registro:




También vemos un componente común en todas las rutas de la página: el *footer*.

En él podemos ver una breve descripción de la empresa bajo el título de *"¿Quienes somos?"*, bajo el título de "*Encuéntranos aquí*" tenemos un mapa en el que al clicar nos llevará a su enlace en *Google Maps,* y a la derecha, bajo el título de *"Contáctanos"*, los datos de contacto el administrador: "*Whatsapp"*, que tiene un enlace directo al chat con el número de teléfono asociado, y "*Llámanos"*, que lleva directamente a llamada de teléfono.

## Anuncios Page

A continuación, mostraremos la página en la que aparecen todos los anuncios:

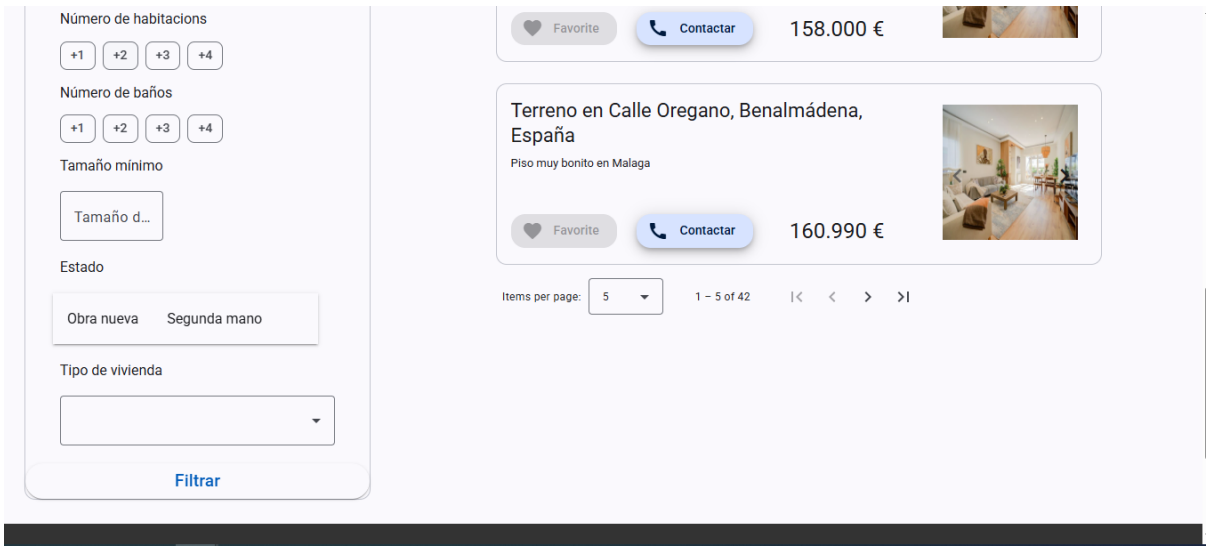


En esta página se ven todos los anuncios publicados en la base de datos.

A la izquierda de la página tenemos una columna de filtros, en los que se pueden buscar anuncios según las preferencias que marque el usuario.

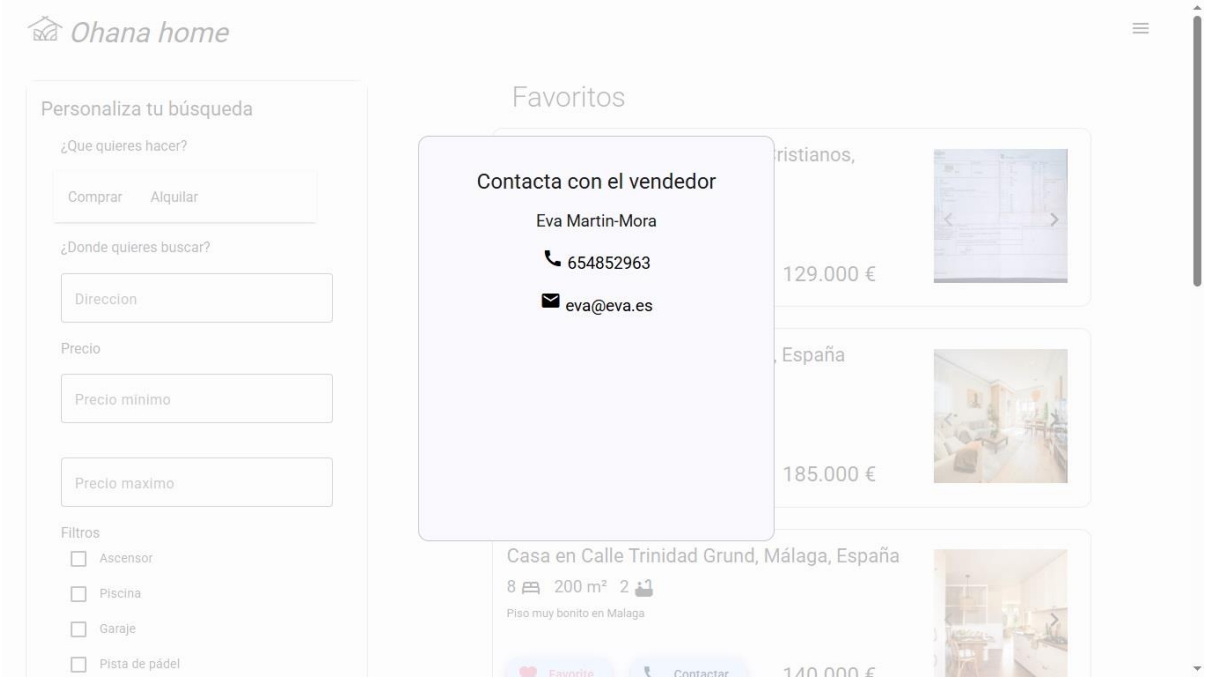
A la derecha de la página, se ven los anuncios publicados (con o sin filtros). Se puede ver de forma resumida los detalles de cada anuncio. Se puede ver también la opción de marcar como favorito; en caso de que el usuario logueado ya le haya dado previamente, aparecerá en color rosa, y al volver a clicar volvería a color azul.

En caso de no haber un usuario logueado, queda inhabilitada esta función:




En la parte baja de esta página, vemos que para acceder a los filtros seleccionados hay que dar click al botón "*Filtrar*".

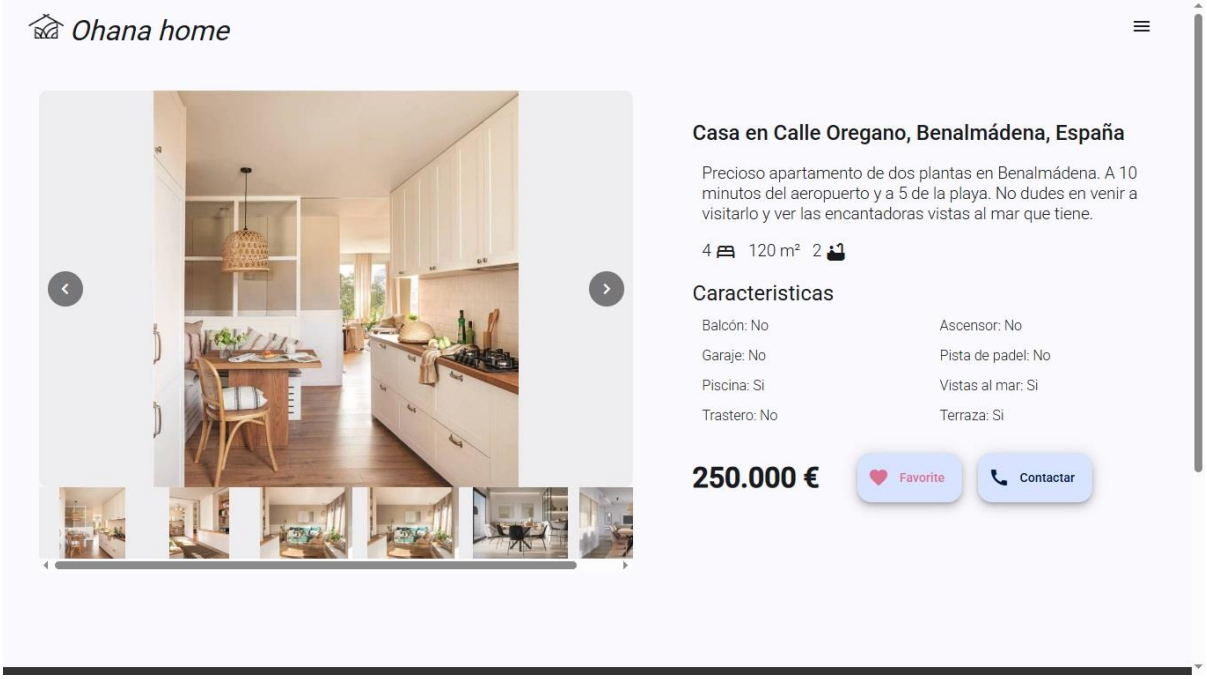
También tenemos la opción de "*Contactar*". Ésta nos llevará a un *overlay* con los datos de usuario que publicó el anuncio, es decir, el vendedor:




Al clicar el título, se accederá a la información completa de dicho anuncio.

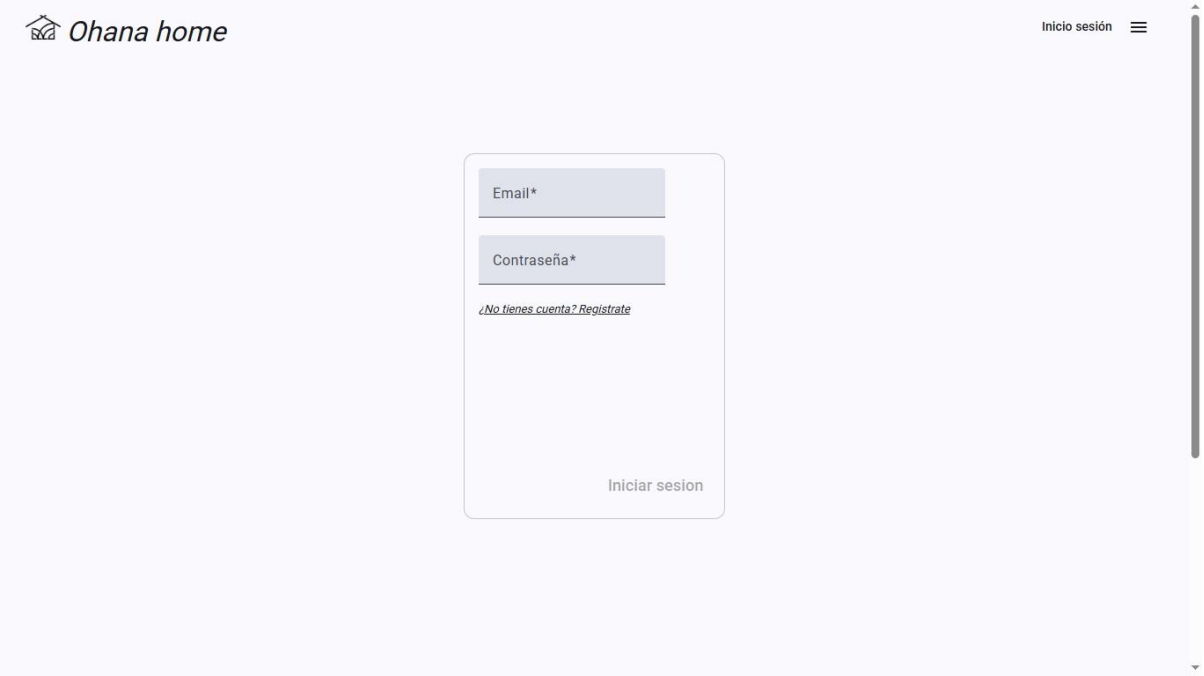
## Anuncio Page

En esta ruta, el usuario podrá ver todos los detalles de un anuncio en específico. Incluye una galería de imágenes destacadas con un sistema de navegación para visualizar diferentes fotos de la propiedad. Al igual que en la página de anuncios, tenemos la opción de dar click tanto a favoritos como a contacto, que tendrá el mismo efecto que si lo hace desde la página anterior.


Ohana home
Casa en Calle Oregano, Benalmádena, España
Precioso apartamento de dos plantas en Benalmádena. A 10 minutos del aeropuerto y a 5 de la playa. No dudes en venir a visitarlo y ver las encantadoras vistas al mar que tiene.
4 120 m² 2
Caracteristicas
Balcón: No
Garaje: No
Piscina: Si
Trastero: No
Ascensor: No
Pista de padel: No
Vistas al mar: Si
Terraza: Si
250.000 €
Favorite
Contactar

## Inicio sesión Page

Desde aquí podremos hacer *Log-in* en nuestra aplicación.



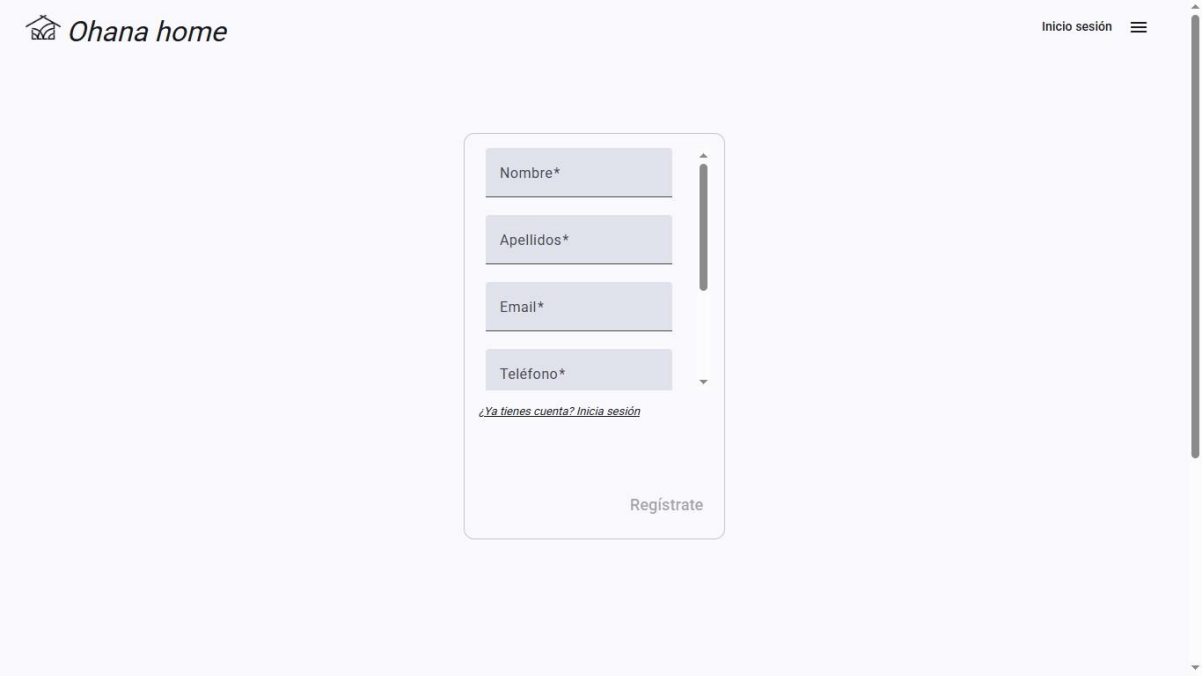

Esta página se encarga de recoger el email y la contraseña del usuario, y compararla con alguna existente en la base de datos. En caso de ser exitoso el inicio de sesión, el back se encargará de guardar un token en el localhost, que recoge, entre otras cosas, el email del usuario. De esa forma, éste puede ser consultado en cualquier momento, pudiéndonos facilitar la información del usuario que está registrado en ese momento.

Una vez iniciada la sesión, nos llevará a la página donde estuviésemos previamente.

En caso de no tener cuenta, nos da la opción a navegar hasta la página de *registro* para dar de alta un nuevo usuario.

## Registro Page

En esta página nos podemos registrar como nuevo usuario.




Tenemos unos requisitos previos que han de ser cumplidos para poder finalizar el proceso:

- El email no debe existir previamente en la base de datos
- La contraseña debe ser fuerte, es decir, debe tener al menos una mayúscula, una minúscula, un carácter numérico y un carácter especial (como: @$!%*?&./,).
- La contraseña hay que introducirla dos veces, y la segunda ha de coincidir con la primera.
- Todos los campos son obligatorios.

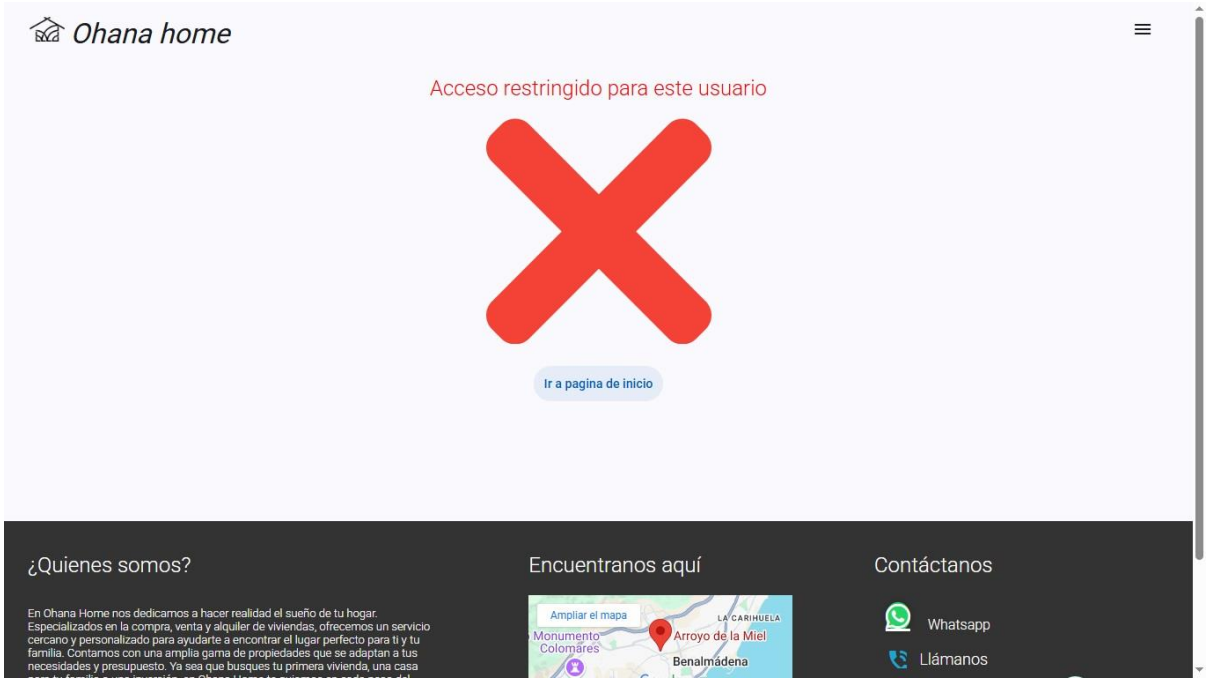
Una vez que el usuario pueda registrarse correctamente, se hará un login automático, lo que nos llevará la página donde estuviésemos previamente.

# Parte privada

## Mi perfil Page

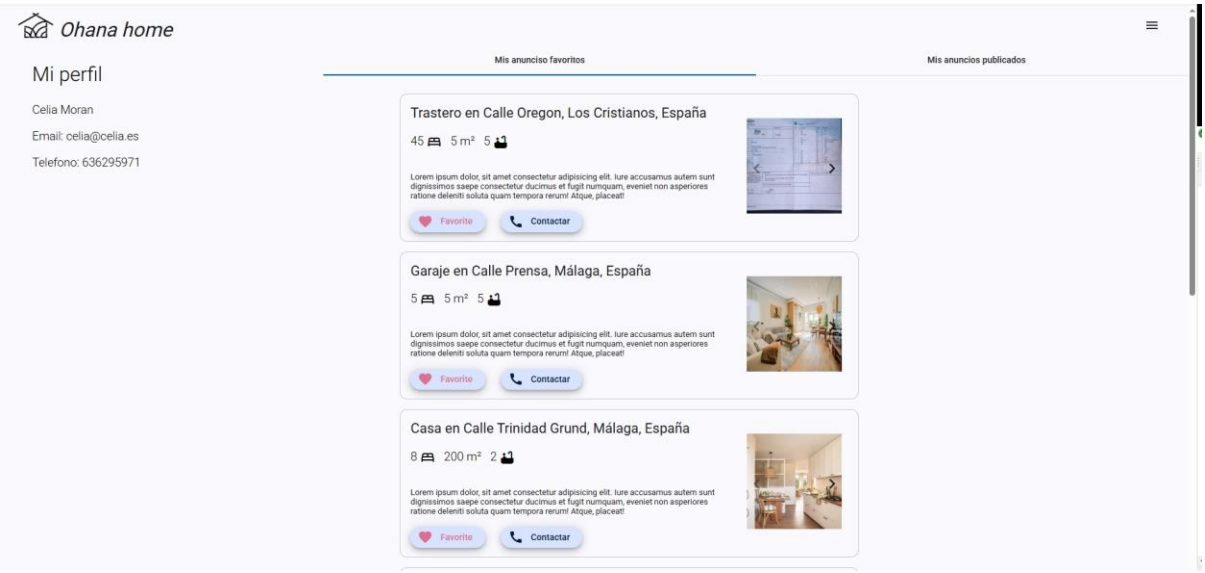
Esta página estará protegida con un *Guard* (*AuthGuard*) para que solamente se pueda acceder a ella en caso de que el usuario haya iniciado sesión previamente.

También tenemos otro *Guard* (*ValidarUsuarioGuard* ) en el que no nos permite la entrada en caso de querer acceder por la url a un perfil que no es el nuestro. En ese caso nos llevará a la *Restricted Page:*




En ella se pueden ver los datos básicos del usuario registrado. Así como sus anuncios publicados y sus favoritos. En los favoritos, en caso de desmarcar alguno, se eliminaría automáticamente.

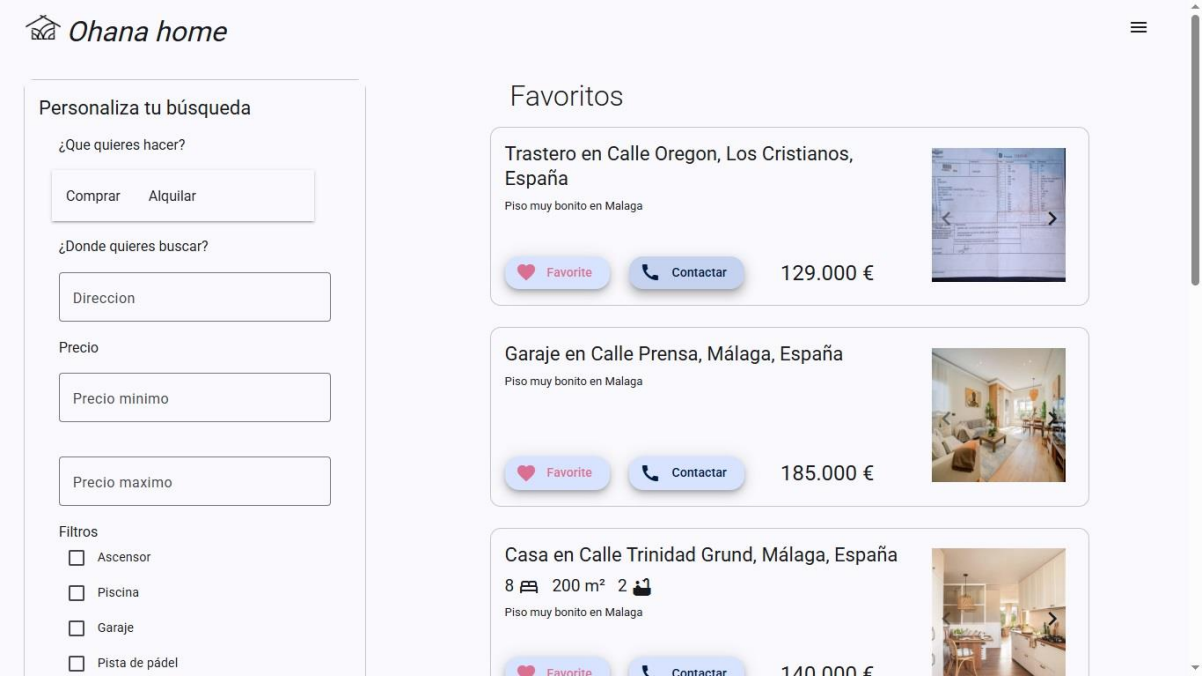
En caso de querer eliminar algún anuncio sería desde esta página.




Al pulsar tanto al email como al teléfono, lleva al usuario tanto a enviar un email a esa dirección de correo, como a llamar al teléfono.

## Mis Favoritos Page

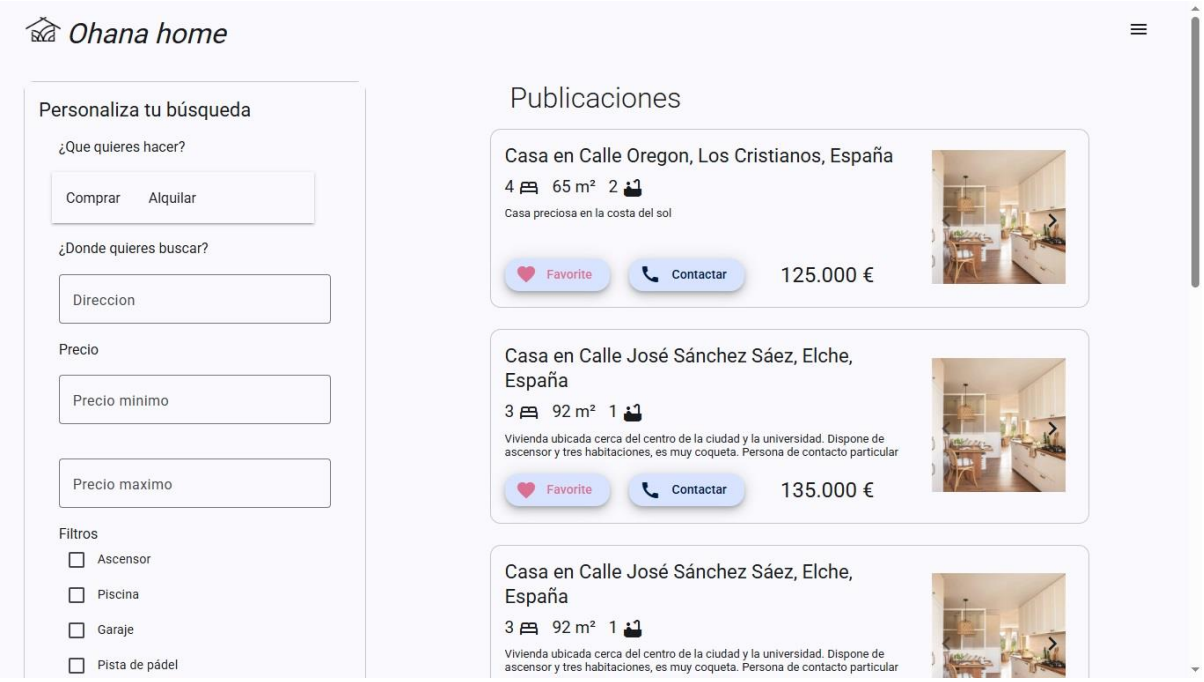
En esta página se muestran los anuncios favoritos del usuario. Es otra página está protegida con los *Guards*. Tiene también activos el *AuthGuard y el ValidarUsuarioGuard,* como antes expliqué.




Al desmarcar el favorito, se quitaría instantáneamente de la vista.

## Mis Publicaciones Page

En esta página se muestran las publicaciones del usuario. Al igual que en "*Mis Favoritos Page",* tienen activos los dos *Guards* que tiene el proyecto.

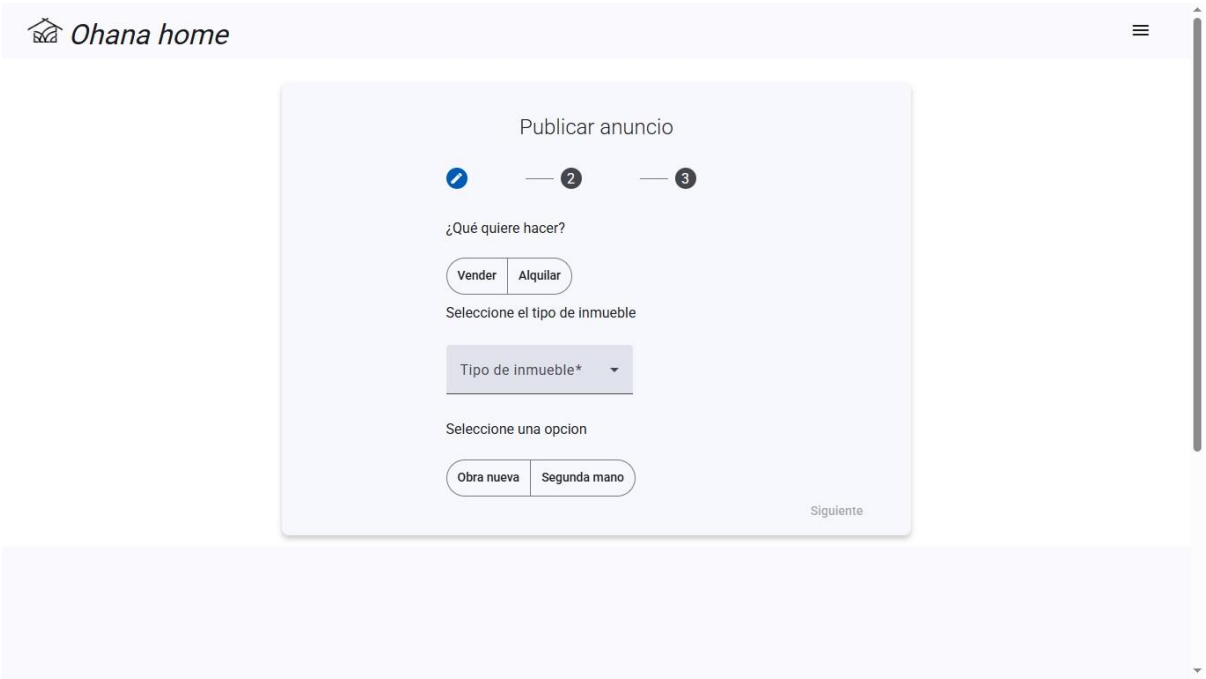


## Publicar anuncio Page

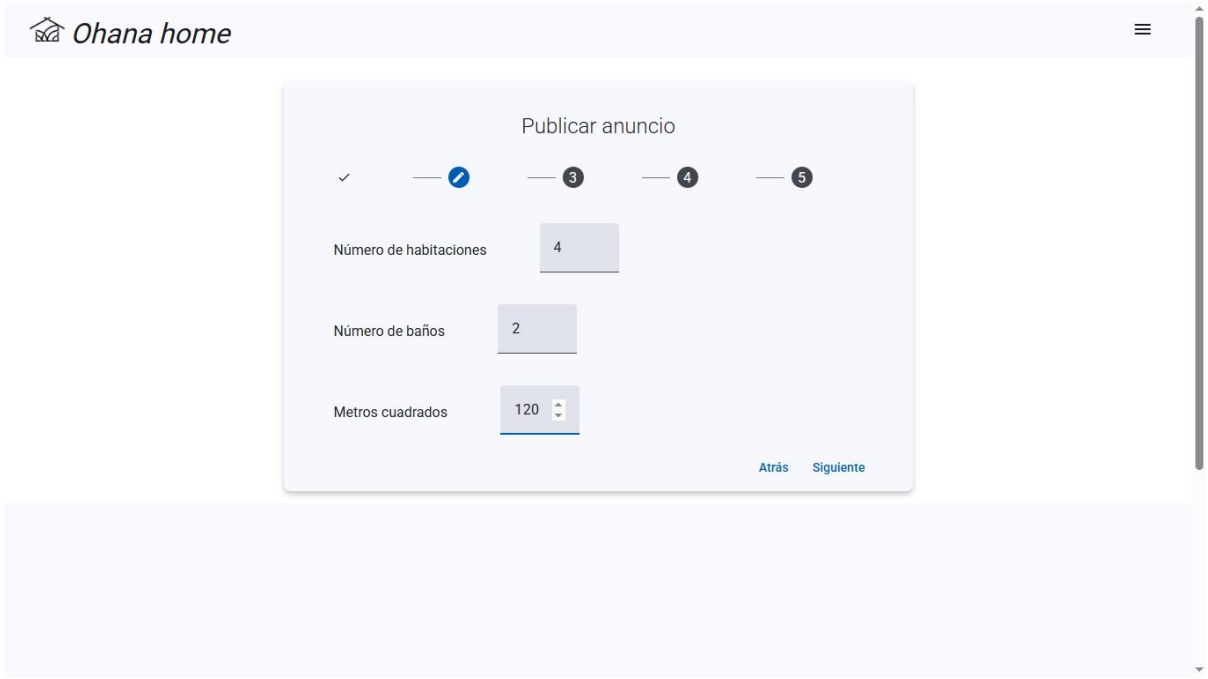
Esta página está restringida al usuario logueado. Se utiliza para publicar un nuevo anuncio.  Se hace a través de un *Mat-Stepper* con cinco pasos.

Paso 1:

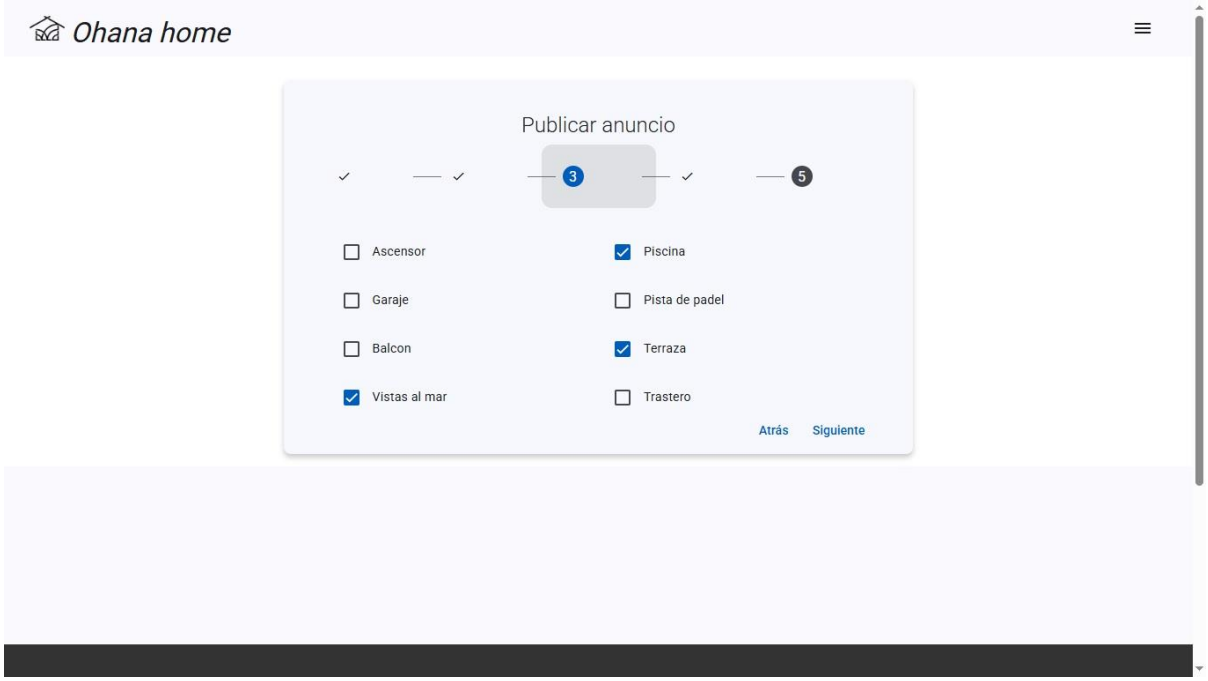
Tiene un condicional, por lo que en caso de que el *Property Type* sea terreno, trastero o garaje, se saltará los pasos dos y tres, ya que dejaremos como null las propiedades que no son necesarias para este tipo de propiedades (como número de habitaciones, baños, si tiene o no ascensor o balcón, etc..).




Paso 2:


Ohana home
Publicar anuncio
3
4
5
Número de habitaciones
4
Número de baños
2
Metros cuadrados
120
Atrás
Siguiente

Paso 3:

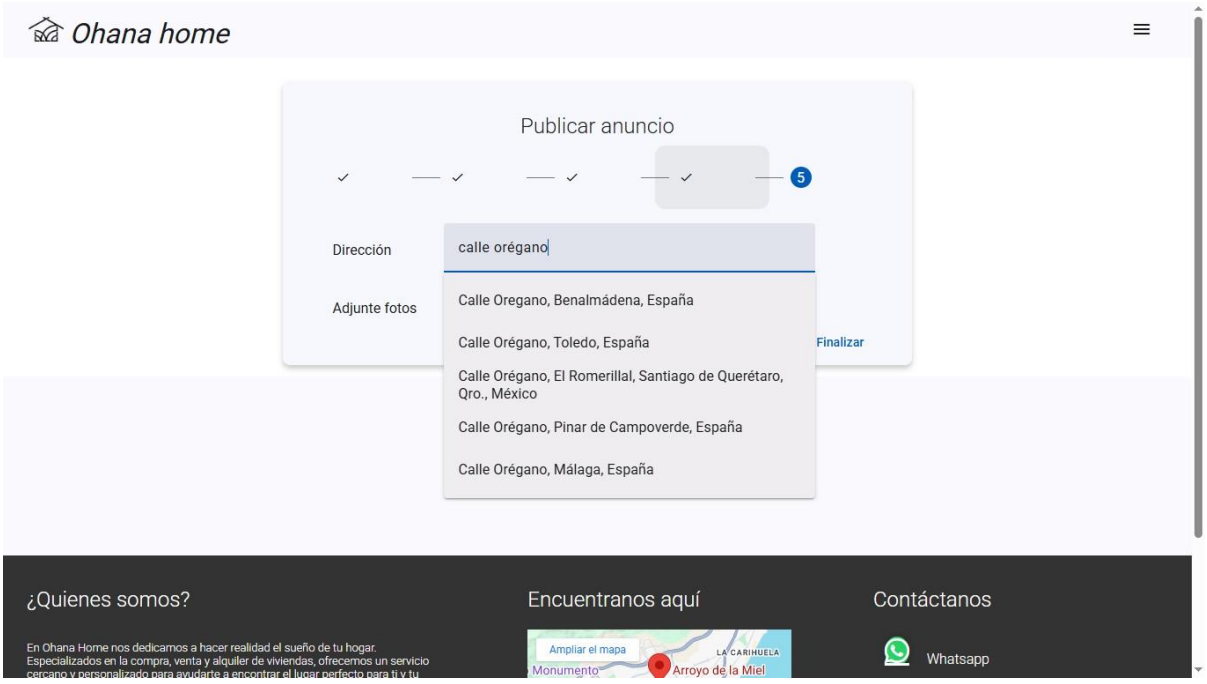

Ohana home
Publicar anuncio
3
5
Ascensor
Piscina
Garaje
Pista de padel
Balcon
Terraza
Vistas al mar
Trastero
Atrás
Siguiente

Paso 4:

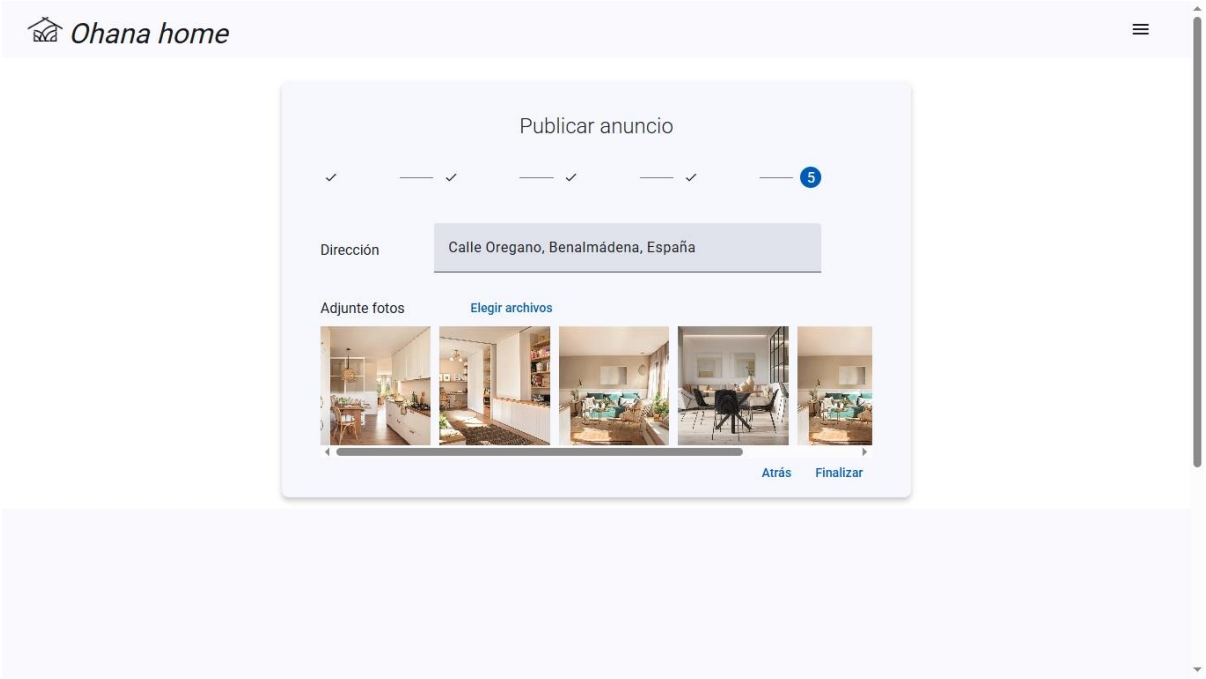



Paso 5:

Está integrada la librería de *Google Maps* para que pueda cargar las direcciones existentes en todo el mundo.

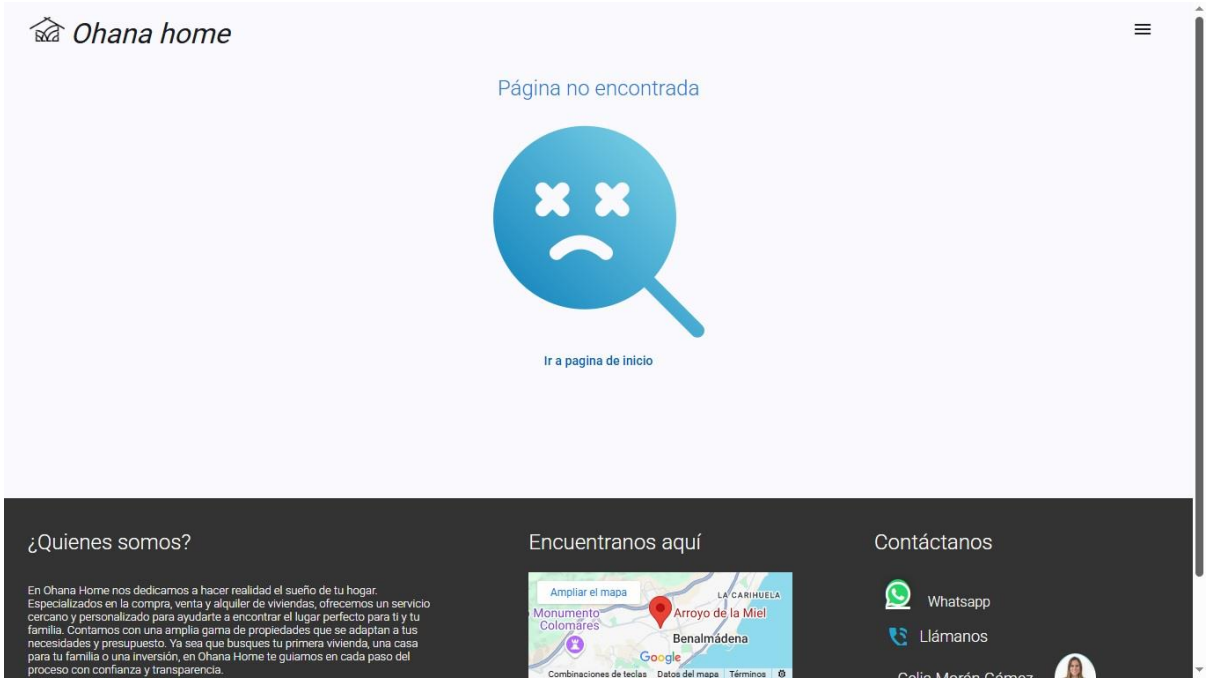



Al adjuntar las imágenes asociadas a un anuncio, se pueden previsualizar antes de ser enviadas:


Ohana home
Publicar anuncio
5
Dirección
Calle Oregano, Benalmádena, España
Adjunte fotos
Elegir archivos
Atrás
Finalizar

## Not Found Page

En caso de tratar de acceder a una url que no tiene ninguna ruta asociada, nos llevará a esta página. Tiene un botón con un *navigate* que nos llevará a la *Home Page*.



# 6. Mejoras

El proyecto ha cumplido con las expectativas que se planearon desde un principio. No obstante, han ido surgiendo posibles mejoras para sacar un mayor rendimiento a la aplicación.

Esta aplicación deja puertas abiertas a ser mejorada, y a proveer al cliente de una mejor experiencia al utilizarla, pero se dejarán para versiones futuras de la misma.

## Búsqueda por mapa

La aplicación ya utiliza la librería de *Google Maps* para introducir direcciones de los inmuebles, por lo que ya estaría predispuesta para poder situarlos en un mapa y que de esta forma el usuario pueda buscar navegando por éste.

Esto supondría, aparte de visualizar los resultados en un listado, visualizarlos también en un mapa interactivo.

También sería muy útil integrarlo al final de cada anuncio, viendo así, la ubicación de cada anuncio.

## Simular hipoteca

Para el usuario final, sería muy útil poder simular cómo quedaría su hipoteca. Esto lo situaríamos en cada anuncio, en el que se pueda introducir el ahorro que se quiera aportar, siendo el mínimo del 20%, el tipo de interés con el que quiere simular, el plazo que desea poner y el propio anuncio arrastrará su precio para poder hacer los cálculos de la cuota mensual.

Este simulador, aparte de estar en cada anuncio, estará también en una ruta aparte. Esto es para poder cotizar una hipoteca sin necesidad de que esté vinculada a un anuncio en concreto.

Además, el simulador podría ofrecer información adicional sobre gastos asociados, como impuestos y comisiones, para una estimación más realista.

## Noticias

Otra mejora que estaba prevista era una página de noticias relacionada con el sector inmobiliario. Esta sección incluirá artículos sobre tendencias del mercado, consejos para compradores y vendedores, así como información sobre normativas y financiamiento.

Además, contará con noticias de última hora sobre cambios en la industria inmobiliaria y análisis de expertos. La idea es mantener a los usuarios informados con contenido relevante y actualizado. De esta manera, la plataforma no solo servirá para la compra y

venta, sino también como una fuente confiable de información para quienes buscan tomar decisiones bien fundamentadas.

## Responsive design

Garantizar que la aplicación se adapte a diferentes tamaños de pantalla y dispositivos (móviles, tabletas y escritorios), mejorando la experiencia de usuario y optimizando el rendimiento en dispositivos con recursos limitados.

## Mostrar anuncios eliminados

Implementar una funcionalidad que permita visualizar aquellos anuncios que han sido eliminados, sin afectar la experiencia del usuario final en la navegación de anuncios activos. De esta forma, se podrían recuperar anuncios eliminados.

## Optimizar el front-end

Aunque no supondría como tal una mejora en la aplicación desde el punto de vista del usuario, sería conveniente optimizar la parte del cliente con los conocimientos adquiridos.

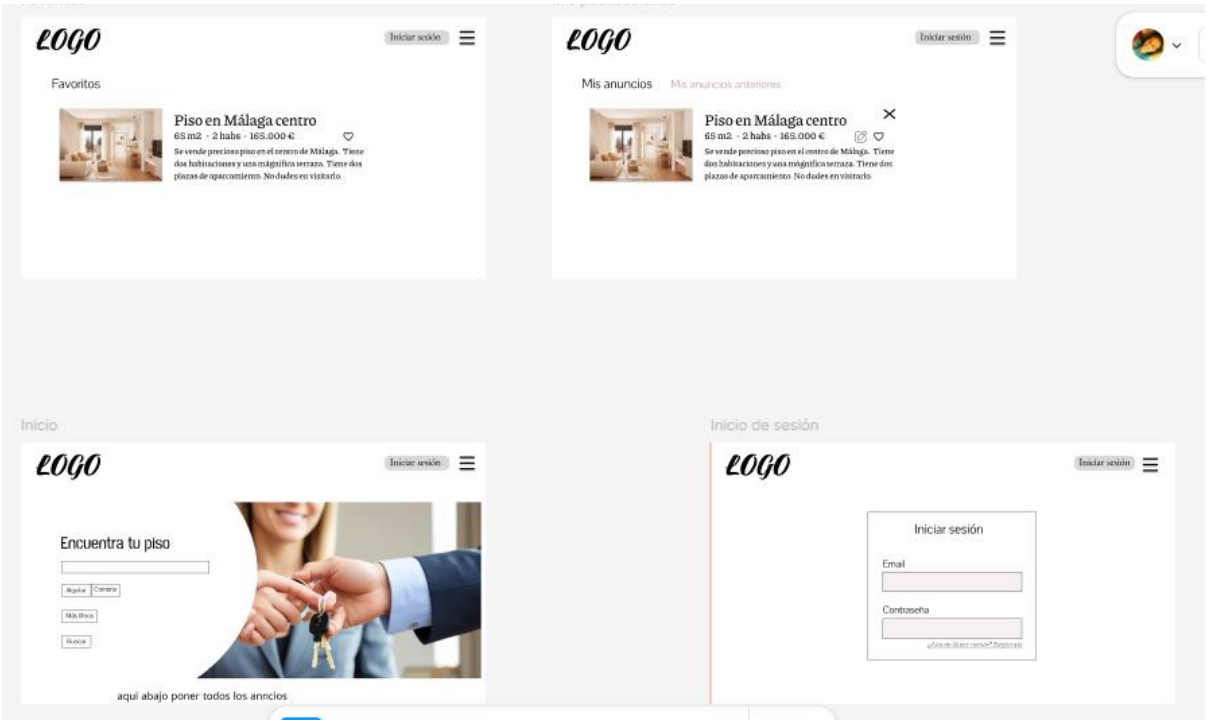
Un ejemplo de esto es utilizar más componentes, como por ejemplo un componente para los anuncios, es decir, los *mat-cards* utilizados para cada anuncio que sean reutilizables.

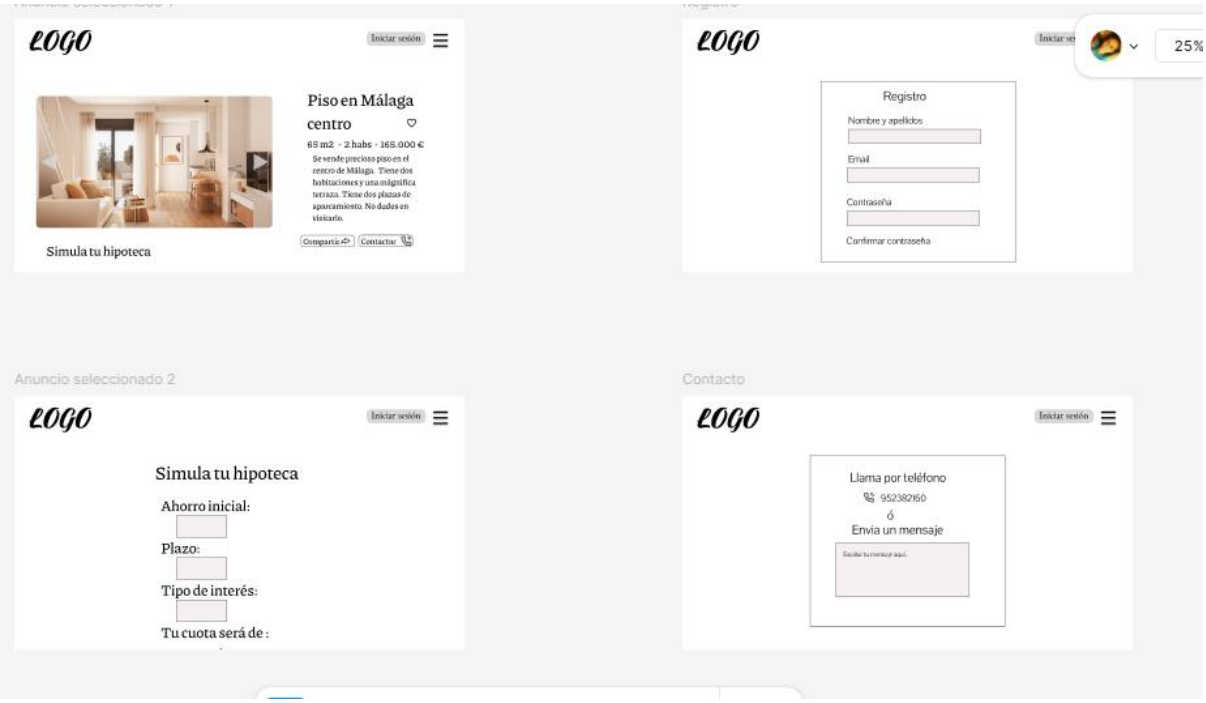
# 7. Recursos

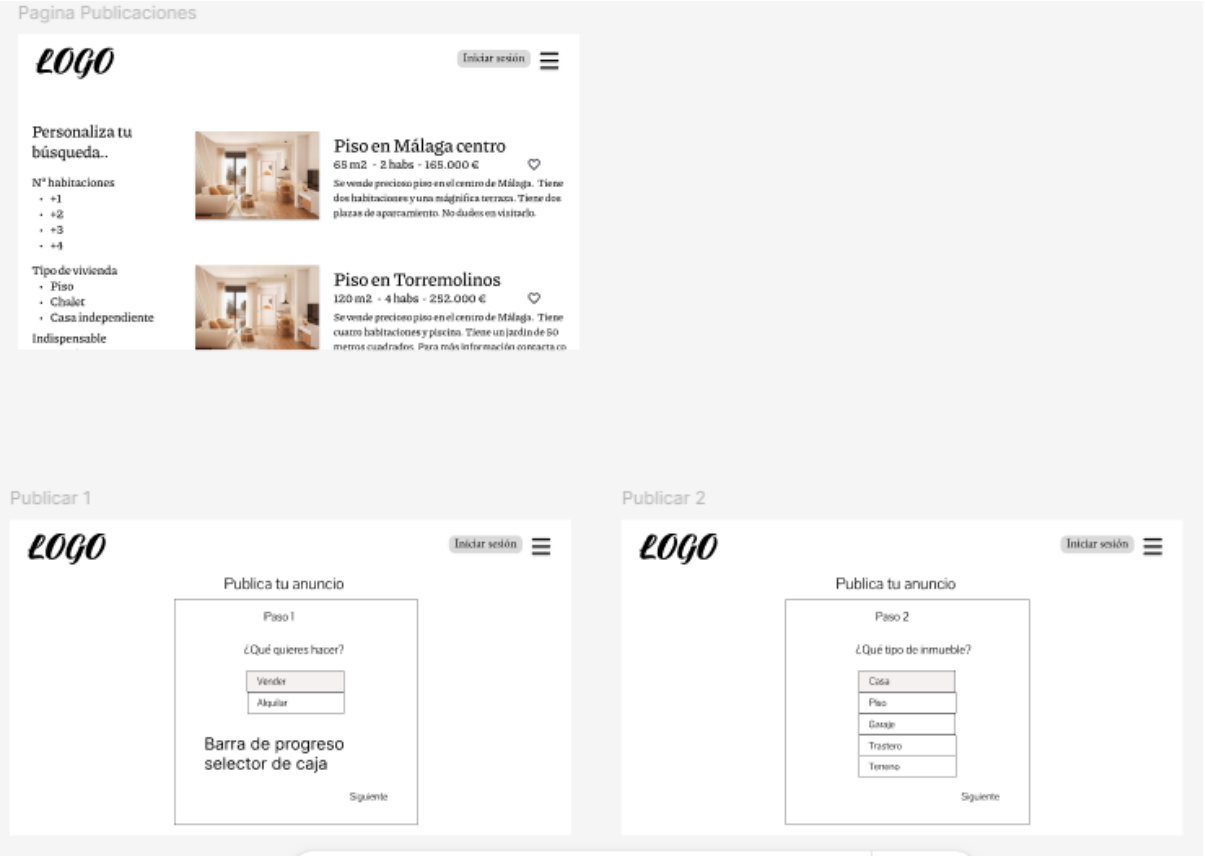
En este apartado veremos los recursos utilizados durante la creación del proyecto.

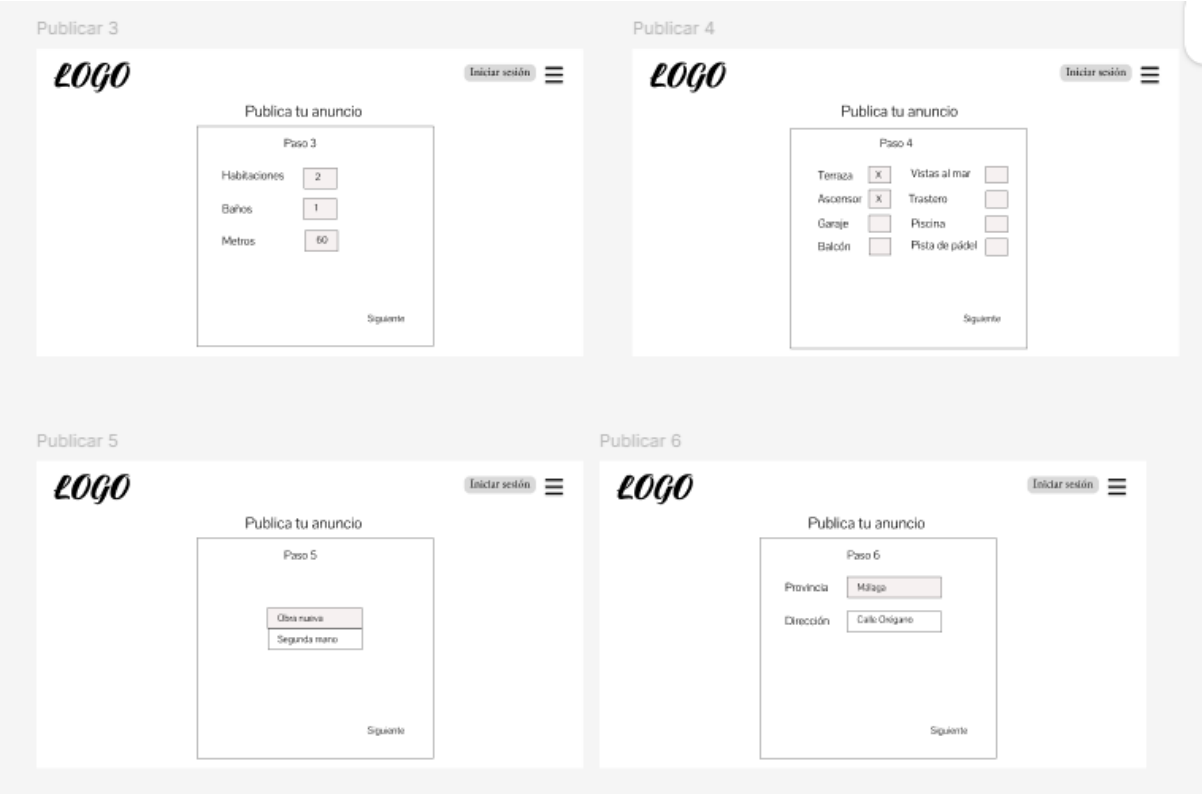
## Figma

*Figma* se utilizó para diseñar prototipos interactivos, revisar y validar ideas visuales antes de pasar a la implementación. Esto me sirvió para tener la idea clara de lo que quería lograr con la aplicación antes de ejecutarla, y poder así tener marcados los objetivos y evitar la improvisación. Esto garantizó que el diseño final fuera intuitivo, atractivo y coherente en todos los componentes de la plataforma.




LOGO
Iniciar sesión
Piso en Málaga centro
65 m2 - 2 habs - 165.000 €
Se vende precioso piso en el centro de Málaga. Tiene dos habitaciones y una mágnifica terraza. Tiene dos plazas de aparcamiento. No dudes en visitarlo.
Compartir
Contactar
Simula tu hipoteca
Registro
Nombre y apellidos
Email
Contraseña
Confirmar contraseña
Anuncio seleccionado 2
Simula tu hipoteca
Ahorro inicial:
Plazo:
Tipo de interés:
Tu cuota será de :
Contacto
Llama por teléfono
952382160
ó
Envía un mensaje


Pagina Publicaciones
LOGO
Iniciar sesión
Personaliza tu búsqueda..
Nº habitaciones
+1
+2
+3
+4
Tipo de vivienda
Piso
Chalet
Casa independiente
Indispensable
Piso en Málaga centro
65 m2 - 2 habs - 165.000 €
Se vende precioso piso en el centro de Málaga. Tiene dos habitaciones y una mágnifica terraza. Tiene dos plazas de aparcamiento. No dudes en visitarlo.
Piso en Torremolinos
120 m2 - 4 habs - 252.000 €
Se vende precioso piso en el centro de Málaga. Tiene cuatro habitaciones y piscina. Tiene un jardín de 50 metros cuadrados. Para más información contacta co
Publicar 1
Publica tu anuncio
Paso 1
¿Qué quieres hacer?
Vender
Alquilar
Barra de progreso selector de caja
Siguiente
Publicar 2
Publica tu anuncio
Paso 2
¿Qué tipo de inmueble?
Casa
Piso
Garaje
Trastero
Terreno
Siguiente




También se ha utilizado *Figma* para realizar los *Diagramas UML*. Se ha utilizado para realizar los *casos de uso*, los *diagramas de secuencia*, y los *diagramas de clases*.

En enlace a mi proyecto en *Figma* es:

https://www.figma.com/design/BGGqdI3LJgNfP78NX3mQpN/Proyecto-TFM?node-id=0-1&m=dev&t=x3gFF4sgasQY6Yeg-1

## Postman

*Postman* fue esencial para el desarrollo y testeo de la API. Permite enviar solicitudes *HTTP*, simular diferentes escenarios de uso y verificar las respuestas del backend.

Es una buena forma de poder saber si un error proviene de la parte del cliente o del servidor. Gracias a esta herramienta, se pudo depurar y ajustar los *endpoints* de forma rápida, asegurando que la comunicación entre el cliente *Angular* y el servidor *Spring Boot* funcionara correctamente y sin contratiempos.




## MySQL Workbench

Con *MySQL Workbench* se diseñó, visualizó y administró el esquema de la base de datos. Se definieron tablas, relaciones y se ejecutaron consultas SQL para validar la integridad de los datos. Esta herramienta facilitó la gestión de la base de datos, asegurando un rendimiento óptimo y la correcta implementación de las restricciones de integridad referencial.

Esta herramienta también nos sirve para ir viendo en tiempo real la situación de la base de datos, de esta forma podemos ver si el funcionamiento de nuestra aplicación es correcto o no.




## Spring Tool Suite 4

*Spring Tool Suite 4* fue el entorno de desarrollo elegido para construir el *backend* con *Spring Boot*. Su integración con *Maven* y soporte para el desarrollo rápido de aplicaciones permitió gestionar dependencias, implementar lógica de negocio y probar los servicios de forma ágil. Esto resultó fundamental para el desarrollo de una API robusta y escalable.



## Visual Studio Code

*VS Code* se empleó para desarrollar el *frontend* en *Angular*. Este editor, altamente configurable y con una gran cantidad de extensiones, facilitó la escritura de código en *TypeScript, HTML y CSS*. Su integración con herramientas de control de versiones y depuración permitió un flujo de trabajo eficiente y colaborativo, mejorando la productividad durante el desarrollo de la interfaz.




## GitHub

*GitHub* se utilizó como plataforma de control de versiones para gestionar el código del proyecto. Permite el seguimiento detallado de los cambios realizados y la integración continua. Esto facilitó la gestión de ramas, asegurando la estabilidad y evolución del proyecto a lo largo del tiempo.

Enlace a mi proyecto en GitHub: https://github.com/celia2mlg/mi-proyecto/




## Angular Material

*Angular Material* se integró para crear una interfaz de usuario moderna y coherente mediante el uso de componentes predefinidos, como tarjetas (*mat-cards*), botones, paginadores y formularios. Esto no solo aceleró el proceso de desarrollo, sino que garantizó consistencia visual y accesibilidad. Además, *Mat Icons* permitió incluir

iconografía vectorial clara y escalable, mejorando la comunicación visual de acciones y estados en la aplicación.

